

TRABAJO FIN DE MÁSTER

MÁSTER EN SEGURIDAD INFORMÁTICA
(Ciberseguridad)

ForenAlyze: Plataforma modular para la recogida y análisis automatizado de evidencias forenses digitales

AUTOR: Francisco Javier Molina Rojas

Puerto Real, marzo de 2026

TRABAJO FIN DE MÁSTER

MÁSTER EN SEGURIDAD INFORMÁTICA
(Ciberseguridad)

ForenAlyze: Plataforma modular para la recogida y análisis automatizado de evidencias forenses digitales

DIRECTOR: Juan Boubeta Puig
CODIRECTOR: Jesús Rosa Bilbao
AUTOR: Francisco Javier Molina Rojas

Puerto Real, marzo de 2026

Agradecimientos

A mi madre y a mi padre, por siempre confiar en mí, animarme a seguir formandome y estar siempre a mi lado.

A mi hermano y a mi pareja Isabel, por aguantarme, acompañarme y hacerme reír cuando lo necesitaba.

A mi prima Miriam, por ayudarme en todo lo que hago y estar siempre pendiente de mí.

A mis tutores Juan y Jesús, por ofrecerme realizar esta investigación con ellos y ayudarme siempre que lo he necesitado.

A mis compañeros Juan, Mari, Blanca, Adriana, Isabel, Jorge, Duván, Ale, Paula y Manu, por acompañarme y por apoyarme siempre.

A mis amigos, por ayudarme a desconectar y animarme.

A Ian y a María, por todas las clases de inglés recibidas que tanto me han ayudado a lo largo de mi vida.

A mi profesora Blanca, por apoyarme a no dejar bachillerato y enseñarme que con esfuerzo siempre se puede.

A todos los docentes del máster, por todo el conocimiento que me han transmitido.

Resumen

En los últimos años el volumen y la sofisticación del *malware* se han incrementado de forma notable; especialmente en formatos cotidianos como documentos ofimáticos, imágenes, audio y ejecutables. Este escenario afecta directamente a usuarios y organizaciones, los cuales se ven obligados a analizar de manera constante ficheros potencialmente maliciosos para proteger sus sistemas. Aunque existen múltiples herramientas especializadas (antivirus, reglas YARA, analizadores de documentos, *sandboxes*), suelen requerir una configuración compleja y no están integradas en una plataforma unificada, lo que dificulta su uso por parte de los analistas.

En este contexto se presenta ForenAlyze, una aplicación web de código abierto orientada al análisis forense automatizado de ficheros. El sistema permite a los usuarios subir ficheros potencialmente sospechosos, que son procesados por un *pipeline* de análisis modular. Dicho *pipeline* integra diversos motores y fuentes de información: antivirus ClamAV, reglas YARA para detección basada en firmas, análisis de macros en documentos Office, detección de esteganografía en imágenes/audio, extracción de texto/metadatos mediante Apache Tika, consultas a VirusTotal y análisis dinámico a través de una *sandbox* externa. A partir de estas señales, ForenAlyze genera un informe detallado y asigna un veredicto global, facilitando la interpretación de los resultados por parte del usuario.

La aplicación incorpora además funcionalidades de gestión de usuarios con roles, sistema de alertas y notificaciones, panel de control con estadísticas agregadas, registro de eventos y control de almacenamiento por usuario. Desde un punto de vista tecnológico, ForenAlyze se ha desarrollado sobre el *framework* Flask, utilizando PostgreSQL como sistema gestor de base de datos y ofreciendo un despliegue reproducible mediante Docker y Docker Compose. El proyecto incluye pruebas automatizadas, integración continua y un conjunto de ficheros de ejemplo y reglas YARA procedentes de repositorios públicos, lo que permite validar el comportamiento del sistema en escenarios realistas.

Por tanto, ForenAlyze proporciona una herramienta accesible que unifica en una sola plataforma distintas técnicas y herramientas de análisis forense, agilizando el proceso de análisis. Asimismo, el diseño modular del *pipeline* y la arquitectura abierta del proyecto sientan las bases para futuras ampliaciones como la incorporación de modelos de *machine learning* para la clasificación de *malware*, o la integración con nuevas fuentes de inteligencia de amenazas.

Palabras clave: *Malware*, Análisis forense, Ficheros maliciosos, YARA, ClamAV, VirusTotal, *Sandbox*, Hybrid Analysis, Flask, Aplicación web, *Logs*, Apache Tika, Esteganografía, *Pipeline*.

Índice general

Índice de figuras	11
Índice de tablas	13
I Prolegómeno	1
1. Introducción	3
1.1. Motivación	3
1.2. Objetivos	3
1.3. Glosario de términos	4
1.4. Organización del documento	5
2. Antecedentes	9
2.1. Contexto	9
2.2. Estado de la técnica	11
3. Plan de gestión de proyecto	15
3.1. Metodología de desarrollo	15
3.2. Tecnologías	15
3.3. Planificación del proyecto	16
3.4. Organización	18
3.5. Costes	18
3.6. Riesgos	19
3.7. Gestión de la configuración	19
3.8. Aseguramiento de calidad	19
II Desarrollo	21
4. Requisitos del Sistema	23
4.1. Objetivos del Sistema	23
4.2. Requisitos funcionales	23
4.3. Requisitos no funcionales	24
4.4. Requisitos de información	25
5. Análisis del Sistema	29
5.1. Modelo Conceptual	29
5.2. Modelo de Casos de Uso	30
5.3. Modelo de Interfaz de Usuario	35
5.4. Modelo de Comportamiento	36
6. Diseño del Sistema	41

6.1.	Arquitectura del Sistema	41
6.1.1.	Diseño de alto nivel	41
6.1.2.	Diseño detallado	46
6.2.	Parametrización del software base	48
6.3.	Diseño Físico de Datos	50
6.3.1.	Sistema de archivos locales	50
6.3.2.	Base de datos	50
6.4.	Diseño de la Interfaz de Usuario	51
6.4.1.	Prototipado	51
6.4.2.	Elementos comunes	51
6.4.3.	Página principal	53
6.4.4.	Página de información	54
6.4.5.	Página de inicio de sesión	55
6.4.6.	Página de panel de control (<i>dashboard</i>)	55
6.4.7.	Página de subida de ficheros	56
6.4.8.	Página de ficheros y análisis	58
6.4.9.	Página de informe de análisis	60
6.4.10.	Página de <i>Logs</i>	63
6.4.11.	Página de gestión del almacenamiento	64
6.4.12.	Página de gestión de reglas YARA	64
6.4.13.	Página de gestión de usuarios	65
6.4.14.	Página de edición de perfil	66
7.	Construcción del Sistema	69
7.1.	Entorno de Construcción	69
7.2.	Código Fuente	69
7.2.1.	Detalles de la hoja de estilos	71
7.2.2.	Manejo de datos y creación de gráficos	71
7.3.	Scripts de Base de Datos	71
7.3.1.	Inicialización y creación de usuarios	72
7.3.2.	Limpieza y consistencia de datos	72
8.	Pruebas del Sistema	75
8.1.	Estrategia	75
8.2.	Pruebas Unitarias y Funcionales	75
8.3.	Pruebas de Integración	76
8.4.	Pruebas de Sistema	76
8.5.	Pruebas de Aceptación	77
8.6.	Pruebas de Rendimiento	77
9.	Despliegue del Sistema	81
9.1.	Arquitectura Física	81
9.2.	Instrucciones de despliegue	82
9.2.1.	Ejecución con Python	82
9.2.2.	Virtualización	83
9.2.3.	Requisitos previos	83
9.2.4.	Inventario de componentes	84
9.2.5.	Procedimientos de instalación	84

9.2.6. Pruebas de implantación	85
9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio	86
III Epílogo	88
10.Conclusiones y trabajo futuro	90
10.1. Conclusiones	90
10.2. Lecciones aprendidas	90
10.3. Trabajo futuro	91
Información sobre Licencia	95
Bibliografía	97
IV Anexos	101
A. Manual de usuario	103
A.1. Introducción	103
A.2. Instalación	103
A.3. Uso del sistema	103
A.3.1. Inicio del sistema	104
A.3.2. Inicio de sesión	104
A.3.3. Panel de control y alertas	104
A.3.4. Subida de ficheros	105
A.3.5. Histórico de ficheros y análisis	105
A.3.6. Informe de análisis	105
A.3.7. Gestión de almacenamiento	106
A.3.8. <i>Logs</i> y auditoría	106
A.3.9. Gestión de usuarios	106
A.3.10. Perfil de usuario	107
B. Dependencias del aplicativo	109

Índice de figuras

3.1. Diagrama gantt - estimación de duración de las tareas	17
3.2. Diagrama gantt - duración real de las tareas	17
5.1. Diagrama conceptual de datos	30
5.2. Caso de uso Subir fichero	31
5.3. Caso de uso Crear usuario	32
5.4. Caso de uso Cambiar contraseña	33
5.5. Caso de uso Editar foto de perfil de usuario	33
5.6. Caso de uso Editar regla YARA	34
5.7. Caso de uso Consultar <i>logs</i> de actividad del sistema	34
5.8. Caso de uso Subir nueva regla YARA	35
5.9. Diagrama de navegación	36
5.10. Diagrama de secuencia Subir fichero	36
5.11. Diagrama de secuencia Crear usuario	37
5.12. Diagrama de secuencia Cambiar contraseña	37
5.13. Diagrama de secuencia Editar foto de perfil de usuario	38
5.14. Diagrama de secuencia Editar regla YARA	38
5.15. Diagrama de secuencia Consultar <i>logs</i> de actividad	39
5.16. Diagrama de secuencia Subir nueva regla YARA	39
6.1. Diagrama de contexto	42
6.2. Diagrama de contexto - Interfaz	43
6.3. Diagrama de contexto - Aplicación Flask	43
6.4. Diagrama de contexto - Gestor de sesiones	44
6.5. Diagrama de contexto - Servidor	45
6.6. Diagrama de contexto - <i>Pipeline</i> de análisis	45
6.7. Diagrama de contexto - Base de datos	46
6.8. Barra de navegación superior - usuario no autenticado	52
6.9. Barra de navegación superior - usuario autenticado (administrador) . .	52
6.10. Barra de navegación superior - usuario autenticado (estándar)	52
6.11. Barra inferior	53
6.12. Página principal - Portada	53
6.13. Página principal - Información adicional	54
6.14. Página principal - FAQ	54
6.15. Página de información	54
6.16. Página de inicio de sesión	55
6.17. Página de inicio de sesión - Error	55
6.18. Página de panel de control (<i>dashboard</i>)	56
6.19. Página de subida de ficheros	57
6.20. Página de subida de ficheros - fichero seleccionado	57
6.21. Página de subida de ficheros - error: tipo de fichero no permitido	58
6.22. Página de subida de ficheros - error: supera el tamaño máximo permitido	58
6.23. Página de ficheros y análisis - pestaña Files	59

6.24. Página de ficheros y análisis - pestaña Files (modal de metadatos) . . .	59
6.25. Página de ficheros y análisis - pestaña Analyses	60
6.26. Página de informe de análisis - Información general y veredicto	61
6.27. Página de informe de análisis - Motores y reglas	61
6.28. Página de informe de análisis - Inteligencia externa (VirusTotal y Hybrid Analysis)	61
6.29. Página de informe de análisis - Módulo de análisis de macros	62
6.30. Página de informe de análisis - Módulo de análisis de texto y metadatos (Apache Tika)	62
6.31. Página de informe de análisis - Módulo de esteganografía en imágenes .	62
6.32. Página de informe de análisis - Módulo de esteganografía en audio . . .	63
6.33. Página de <i>Logs</i>	63
6.34. Página de gestión del almacenamiento	64
6.35. Página de gestión de reglas YARA - listado de reglas	65
6.36. Página de gestión de reglas YARA - editor de reglas	65
6.37. Página de gestión de usuarios	66
6.38. Página de edición de perfil	66

Índice de tablas

2.1. Comparativa funcional entre Assemblyline 4, CAPE Sandbox y ForenAlyze	13
3.1. Desglose de costes	18
3.2. Tabla de riesgos	19
8.1. Resultados de las pruebas de rendimiento usando Locust	78
B.1. Tabla de dependencias de ForenAlyze	109

Parte I

Prolegómeno

1. Introducción

En este capítulo se presenta la motivación que ha impulsado la realización del proyecto, así como el alcance y los objetivos que lo definen. Asimismo, se incluye el glosario de términos necesarios para su correcta comprensión y se describe la estructura del documento, con el fin de guiar al lector a lo largo del trabajo.

1.1. Motivación

En la actualidad, el gran incremento de dispositivos digitales y de servicios en la nube ha provocado un crecimiento masivo en la cantidad de evidencias digitales que deben ser preservadas y analizadas. Este aumento, junto con la especialización de las amenazas (*malware*, campañas de *phishing*, documentos maliciosos y técnicas de ocultación como la esteganografía), hace que las organizaciones y los analistas forenses afronten un proceso de análisis más complejo y costoso ([Michelet et al., 2023](#)).

Las investigaciones forenses modernas necesitan tratar con múltiples fuentes y formatos como documentos Office y PDF, ejecutables, imágenes, audios... Así como la combinación de técnicas estáticas y dinámicas para conseguir conclusiones fiables. Además, existen gran cantidad de herramientas especializadas que tienen como objetivo solucionar problemas específicos, lo que dificulta los flujos de trabajo automatizados y humanos ([Pandey y Mehtre, 2014](#)).

También, la utilización de servicios externos para la obtención de datos de inteligencia, plantea desafíos en la privacidad y la custodia de las evidencias.

Frente a este escenario, surge una necesidad de soluciones que centralicen y automaticen la recogida, el análisis y la gestión de evidencias, permitiendo al analista obtener resultados sin perder las garantías forenses ni la protección de datos.

Para dar respuesta a esta necesidad, este Trabajo Fin de Máster (TFM) propone ForenAlyze, una plataforma modular que facilita la utilización de múltiples técnicas de análisis, la visualización de resultados y la generación de evidencias trazables y exportables, para su uso en contextos académicos y de laboratorio.

1.2. Objetivos

El objetivo general de este Trabajo Fin de Máster es desarrollar una plataforma modular para la recogida, análisis automatizado y gestión de evidencias forenses digitales, que integre técnicas estáticas y heurísticas (YARA, antivirus, esteganografía LSB/estadística, detección de macros, análisis de audio y extracción textual); y que proporcione una interfaz web usable para analistas y forenses.

Para abordar este objetivo general, se definen los próximos objetivos específicos:

1. Diseñar una arquitectura modular que permita integrar y utilizar distintas técnicas de análisis forense de forma desacoplada y extensible.
2. Implementar un sistema de recogida de evidencias digitales que permita la subida, el almacenamiento controlado y la trazabilidad de ficheros asociados a usuarios autenticados.
3. Incorporar componentes de gestión y auditoría: implementar gestión de usuarios (roles y cuotas), permisos básicos de acceso y operaciones sobre evidencias, y registro de eventos forenses (*logs*) con metadatos relevantes, por ejemplo, *IP* y *user-agent* para preservar la trazabilidad y el soporte en procesos periciales.
4. Automatizar el análisis de evidencias mediante la aplicación de técnicas estáticas y heurísticas para la identificación de:
 - Indicadores de *malware*.
 - Presencia de macros o código embebido sospechoso.
 - Posibles técnicas de ocultación de información (esteganografía).
 - Metadatos relevantes para el análisis forense.
5. Clasificar y correlacionar los resultados del análisis con el objetivo de obtener un veredicto global por evidencia, así como generar alertas asociadas según distintos niveles de severidad.
6. Diseñar una interfaz *web* que permita visualizar de forma clara y estructurada el estado de las evidencias, los resultados del análisis, las alertas generadas y métricas agregadas.
7. Incorporar mecanismos de auditoría y trazabilidad, registrando las acciones relevantes realizadas sobre las evidencias y por los usuarios del sistema.
8. Validar el funcionamiento de la plataforma desarrollada mediante la ejecución de pruebas controladas, evaluando el comportamiento del sistema ante distintos tipos de evidencias y escenarios de uso.
9. Diseñar y documentar el proceso de despliegue y puesta en servicio de la plataforma, analizando los requisitos necesarios para su ejecución en un entorno operativo.

1.3. Glosario de términos

En este apartado se exponen los términos técnicos y siglas de uso frecuente en la memoria, con el objetivo de facilitar la comprensión del lector.

Análisis estático: Técnicas y procesos que permiten analizar un fichero sin necesidad de ejecutarlo, incluyendo extracción de metadatos, firmas (*hashes*) y reglas de detección.

Análisis dinámico: Análisis del comportamiento de un fichero mediante su ejecución en un entorno aislado (*sandbox*).

Apache Tika: Herramienta para la extracción automática de texto y metadatos de documentos como PDF y Office.

ClamAV: Motor antivirus de código abierto utilizado para detectar *malware* de forma automática mediante firmas.

Esteganografía: Técnica de ocultación de información dentro de otros ficheros (imágenes, audio...) sin que sea evidente su presencia.

Hash: Valor único generado a partir del contenido de un fichero utilizando algoritmos (por ejemplo MD5, SHA1, SHA256...) usado para verificar integridad e identificar muestras.

Heurística: Detección basada en patrones o comportamientos sospechosos.

MIME: Identificador utilizado para describir el tipo de contenido de un fichero (por ejemplo, application/pdf o image/jpeg).

Metadatos: Información descriptiva asociada a un fichero, como autor, fechas, tamaño o formato.

Sandbox: Entorno aislado usado para ejecutar y analizar de forma controlada ficheros maliciosos.

Trazabilidad: Capacidad de reconstruir las acciones que han realizado los usuarios en el sistema.

VirusTotal: Servicio de Google que permite consultar el análisis de un fichero mediante múltiples motores antivirus.

YARA: Lenguaje y motor de reglas usado para identificar patrones específicos en ficheros binarios.

1.4. Organización del documento

El documento está organizado en tres partes principales.

- En la primera parte, haremos una introducción del proyecto, exponiendo la motivación, los objetivos y el alcance del mismo. Además, daremos un contexto de la situación actual y especificaremos el plan de gestión usado para el desarrollo del proyecto.
- En la segunda parte, se presentaremos el desarrollo del proyecto, incluyendo la toma de requisitos, el análisis, diseño, construcción, pruebas y despliegue del sistema. Específicamente:
 - En el capítulo 4, se definirán los objetivos del sistema; además de los requisitos funcionales, no funcionales y de información.
 - En el capítulo 5, se hará un análisis del sistema en base a los requisitos del capítulo anterior, definiendo el funcionamiento del mismo mediante diferentes modelos.

- En el capítulo 6, se especificará el diseño del sistema, definiendo los diferentes componentes del proyecto y la relación entre estos usando diagramas de modelo C4. Además, se determinarán las tecnologías y técnicas usadas para la construcción del sistema.
 - En el capítulo 7, se mostrará la construcción del sistema, definiendo el entorno de desarrollo y los ficheros que componen al sistema.
 - En el capítulo 8, se expondrán las pruebas realizadas al sistema.
 - En el capítulo 9, se explicará cómo se ha desplegado el sistema en un servidor de producción.
- En la tercera parte, mostraremos las conclusiones resultantes después de desarrollar el proyecto y expondremos los objetivos alcanzados, así como propuestas de posibles mejoras.

2. Antecedentes

En este capítulo se expone el contexto de la problemática abordada en el proyecto y posteriormente se realiza una revisión del estado de la técnica, comparando nuestra propuesta con otras soluciones existentes.

2.1. Contexto

El análisis forense digital es una disciplina de la ciberseguridad cuya finalidad es la identificación, adquisición, análisis y preservación de evidencias digitales con el objetivo de comprender incidentes de seguridad. Además, se trata de un campo en continua evolución que presenta múltiples desafíos técnicos y metodológicos ([Fakhouri et al., 2024](#); [Guarino, 2013](#)).

Dentro de este ámbito, el *malware* emplea técnicas cada vez más sofisticadas para ocultar su presencia y dificultar su detección; recurriendo a estrategias como la esteganografía en imágenes y audio, mediante la cual se oculta información maliciosa en ficheros aparentemente legítimos a través de modificaciones imperceptibles, como las realizadas sobre los bits menos significativos ([Alenezi et al., 2022](#)) y ([Knöchel y Karius, 2024](#)).

Como respuesta a esta evolución del *malware*, han surgido diversas herramientas y tecnologías orientadas a mejorar su detección y análisis ([Carvey y Altheide, 2011](#)). Entre ellas se encuentran el análisis de metadatos que permite extraer información relevante sobre el origen y las características de los ficheros, el uso de *sandboxes* para la ejecución controlada de muestras sospechosas y los motores antivirus basados en firmas, que contribuyen a un análisis más eficaz y sistemático.

De forma complementaria a los motores tradicionales, las reglas YARA constituyen un lenguaje declarativo muy utilizado en análisis forense para describir patrones de *malware* ([Mahdi y Trabelsi, 2024](#)). Cada regla combina cadenas de texto, secuencias de bytes o expresiones regulares, lo que permite detectar tanto variantes conocidas como comportamientos sospechosos.

Como ejemplo sencillo, en el listado [2.1](#) se muestra una regla YARA básica que busca determinados patrones en documentos Office con macros:

```

rule Office_Macro_FileNet {
    meta:
        description = "Detecta macros que se ejecutan
                        automaticamente y escriben a disco o realizan
                        conexiones HTTP"
    strings:
        $macro    = "Sub AutoOpen"
        $sfso      = "CreateObject(\"Scripting.FileSystemObject
                    \")"
        $net       = /( ?i)https?:\\\/[^\s"]+|MSXML2\.XMLHTTP/
    condition:
        $macro and ($sfso or $net)
}

```

Extracto de código 2.1: Ejemplo de regla YARA sencilla

En esta regla, la sección ‘meta’ contiene una breve descripción de su objetivo. La sección ‘strings’ define los patrones a buscar: ‘\$macro’ identifica macros que se ejecutan automáticamente al abrir el documento, ‘\$sfso’ detecta el uso del objeto ‘FileSystemObject’ para crear o escribir ficheros y ‘\$net’ busca indicios de conexiones de red mediante URLs o el objeto ‘XMLHTTP’.

Finalmente, la sección ‘condition’ establece que el documento coincidirá con la regla si contiene una macro de autoejecución y, además, realiza alguna operación potencialmente sospechosa como escribir en disco o establecer una conexión de red.

Como hemos visto en el capítulo 1, la disciplina de análisis forense digital se ha convertido en una disciplina clave en el mundo de la ciberseguridad actual y en esta han surgido una serie de problemas:

- Problema de complejidad: el análisis forense es complejo, ya que el *malware* puede ocultar su presencia en distintos tipos de ficheros usando técnicas de ofuscación.
- Problema de fragmentación de herramientas: existen diferentes herramientas especializadas para cada tipo de análisis, lo que hace difícil un flujo de trabajo eficiente.
- Problema de automatización: numerosos de los procesos de análisis forense digital son manuales, lo que consume tiempo y recursos.
- Problema de trazabilidad: los analistas forenses deben garantizar la trazabilidad de sus acciones y resultados en todo momento.

Para solventar estos problemas, especificaremos una serie de dimensiones que deben tener en cuenta las soluciones que se propongan, estas son:

- *Pipeline* modular de análisis: permite integrar de diferentes herramientas y técnicas en un flujo de trabajo.
- Soporte de reglas YARA: permite crear y aplicar reglas YARA para la detección de *malware*.

- Soporte para múltiples formatos de archivos: permite analizar distintos tipos de archivos como por ejemplo, imágenes, documentos, ejecutables...
- Soporte para *sandbox*: permite ejecutar ficheros en un entorno controlado para comprobar su comportamiento.
- Obtención de metadatos: permite la extracción de metadatos presentes en los ficheros.
- Detección de macros: permite detectar la presencia de macros en documentos *Office*.
- Detección de técnicas de esteganografía en imágenes y audio: permite detectar la presencia de mensajes ocultos en imágenes y audio.
- Gestión de usuarios y roles: permite gestionar usuarios mediante el uso de roles.
- Auditoría y trazabilidad: se registran las acciones relevantes realizadas por los usuarios del sistema.
- *Dashboard* de visualización de resultados: permite visualizar el estado del sistema y los resultados del análisis.
- Generación de alertas y notificaciones: se generan alertas asociadas a los resultados del análisis.
- Exportación de análisis en formatos estándar: permite exportar los resultados de los análisis en formatos como JSON o PDF.
- Uso académico y de investigación: la plataforma se orienta a un uso académico y de investigación.

El objetivo de este trabajo será desarrollar una plataforma web modular que permita solucionar estos problemas.

2.2. Estado de la técnica

Antes de desarrollar una nueva plataforma, es importante conocer el estado de la técnica en este ámbito.

En la actualidad, existen varias plataformas de análisis forense, que ofrecen distintas funcionalidades. Entre ellas, destacan Assemblyline 4 ([Canadian Centre for Cyber Security, 2020](#)) y CAPE Sandbox ([K. O'Reillyt and A. Brukhovetsky, 2024](#)).

Comenzando por la experiencia de uso de Assemblyline 4 ([Almuhtadi et al., 2021](#)), esta es una plataforma de análisis forense que ha sido desarrollada por el Centro de Ciberseguridad de Canadá (CCCS). Esta plataforma tiene como objetivo principal integrar diferentes herramientas para el análisis de *malware*. Específicamente, ofrece:

- Un *pipeline* modular.
- Da soporte para reglas YARA.
- Admite varios formatos excluyendo audios e imágenes.

- Tiene soporte para *sandbox* usando Cuckoo Sandbox¹.
- Obtención plena de metadatos.
- Tiene soporte para gestión de usuarios, roles y trazabilidad algo limitado debido al uso de ELK².
- Posee la capacidad de integrar dashboards con la limitación de usar Kibana³.
- Notifica a través de un sistema de alertas.
- Está orientada a uso académico y de investigación.

Además de las anteriores limitaciones, esta plataforma también carece de soporte para la detección de técnicas de esteganografía en imágenes o audio, y no soporta la detección de macros.

En cuanto a la documentación oficial de CAPE Sandbox, esta es una plataforma de análisis forense de código abierto que automatiza el análisis de *malware*. Esta ofrece:

- Un *pipeline* modular.
- Da soporte para reglas YARA.
- Admite varios formatos excluyendo audios e imágenes.
- Tiene soporte para *sandbox* usando Cuckoo Sandbox.
- Obtención parcial de metadatos, aunque el historial de análisis se almacena, carece de auditoría para acciones de los usuarios.
- Permite exportar los resultados en formato JSON.
- Está orientada a uso académico y de investigación.

Sin embargo, CAPE Sandbox tiene algunas carencias, como la falta de soporte para la detección de técnicas de esteganografía en imágenes o audio, no soporta la detección de macros, no proporciona un dashboard de visualización de resultados de forma nativa y no tiene un sistema de alertas y notificaciones.

En la siguiente tabla se muestra una comparativa funcional a modo de resumen entre Assemblyline 4, CAPE Sandbox y la solución propuesta en este proyecto, ForenAlyze:

¹Cuckoo Sandbox - <https://cuckoosandbox.org/>

²Elastic Stack - <https://www.elastic.co/es/elastic-stack>

³Kibana - <https://www.elastic.co/es/kibana>

Tabla 2.1 - Comparativa funcional entre Assemblyline 4, CAPE Sandbox y ForenAlyze

Característica	Assemblyline 4	CAPE Sandbox	ForenAlyze
<i>Pipeline</i> modular de análisis	Sí	Sí	Sí
Soporte de reglas YARA	Sí	Sí	Sí
Soporte para múltiples formatos de archivos	Parcial	Parcial	Sí
Soporte para <i>sandbox</i>	Sí	Sí	Sí (integración externa opcional)
Obtención de metadatos	Sí	Parcial	Sí
Detección de macros	No	No	Sí
Detección de técnicas de esteganografía en imágenes y audio	No	No	Sí
Gestión de usuarios y roles	Parcial (ELK)	No	Sí
Auditoría y trazabilidad	Parcial (ELK)	No	Sí
Dashboard de visualización de resultados	Sí (Kibana)	No	Sí
Generación de alertas y notificaciones	Sí	No	Sí
Exportación de análisis en formatos estándar	Sí (JSON)	Sí (JSON)	Sí (JSON y PDF)
Uso académico y de investigación	Sí	Sí	Sí

Este trabajo se orienta a la creación de una plataforma de análisis forense digital que integre las funcionalidades mencionadas anteriormente, con el objetivo de proporcionar una solución más completa y eficiente para los analistas forenses digitales.

3. Plan de gestión de proyecto

En este capítulo se presenta el plan de gestión del proyecto que incluye la metodología de desarrollo, las tecnologías utilizadas, la planificación del proyecto, la organización, los costes, los riesgos, la gestión de la configuración y el aseguramiento de la calidad.

3.1. Metodología de desarrollo

Para el desarrollo de este proyecto se ha dividido en 5 fases principales:

- Planificación del proyecto, recogida de información y documentación: esta fase consiste en la planificación de las tareas a realizar e incluye la recopilación, análisis y organización de la información necesaria junto a la redacción de la documentación.
- Diseño y prototipado: esta fase se centra en el diseño de la UI y UX del aplicativo, así como en la creación de prototipos para visualizar facilitar la implementación.
- Implementación: esta fase se centra en la creación e implementación del software.
- Testeo: esta fase se centra en probar y examinar el software para asegurar su correcto funcionamiento.
- Despliegue: esta fase se centra en desplegar el aplicativo en un servidor de producción.

3.2. Tecnologías

Para construir la solución propuesta se ha empleado el lenguaje de programación Python¹, en su versión 3.12.7. Se ha elegido este lenguaje debido a su versatilidad y a su amplio ecosistema de librerías, que permiten tanto el desarrollo web como el análisis de ficheros y la integración con herramientas de ciberseguridad, lo que lo convierte en una elección adecuada para el proyecto.

Para la sección web del aplicativo, se ha usado el *framework* Flask² junto con el motor de plantillas Jinja2³. Además se ha utilizado Bootstrap 5⁴ y Chart.js,⁵ para crear una UI moderna, responsiva e interactiva.

¹Python 3.12.7 - <https://www.python.org/downloads/release/python-3127/>

²Flask - <https://flask.palletsprojects.com/>

³Jinja2 - <https://jinja.palletsprojects.com/>

⁴Bootstrap - <https://getbootstrap.com/>

⁵Chart.js - <https://www.chartjs.org/>

Para la persistencia de datos, se ha usado Flask-SQLAlchemy que permite definir los modelos de forma simple. Como SGBD se ha seleccionado PostgreSQL⁶. La autenticación y la gestión de sesiones de usuarios se implementan con la extensión Flask-Login, que proporciona funciones para el inicio y cierre de sesión, así como para la protección de *endpoints*.

Para el ámbito de análisis forense se han utilizado diversas librerías. El cálculo de hashes y el tratamiento básico de ficheros se realiza con las bibliotecas estándar de Python, el análisis antivirus local usa ClamAV⁷ mediante el comando `clamscan`, la detección basada en firmas se implementa a través de reglas YARA⁸ mediante la librería `yara-python`⁹, para la detección de macros maliciosas en documentos Office se ha usado la librería `oletools`¹⁰.

Para la extracción de metadatos y contenido de ficheros, se hace uso de Apache Tika¹¹ desplegado como servicio externo que permite extraer texto y metadatos de documentos PDF y Office. Para ficheros de audio se usa la librería Mutagen¹², mientras que el tratamiento de imágenes y la lectura de metadatos EXIF usan la librería Pillow.¹³ La generación de espectrogramas y ciertos análisis numéricos se realizan con NumPy¹⁴ y Matplotlib¹⁵.

La comunicación con servicios de inteligencia, como VirusTotal o Hybrid Analysis, se lleva a cabo mediante el uso de la librería Requests¹⁶ que realiza llamadas a APIs HTTP. Esto permite complementar el análisis local con información reputacional y resultados de análisis dinámico.

Finalmente para facilitar el despliegue, se han utilizado Docker¹⁷ y en concreto Docker Compose. A través de estos, se generan tres contenedores independientes. El primero es para la base de datos PostgreSQL, el segundo es el servidor de Apache Tika y el tercero es la propia aplicación ForenAlyze (ejecutada sobre Gunicorn¹⁸ con ClamAV instalado).

3.3. Planificación del proyecto

Al inicio del proyecto se realizó una planificación inicial, estableciendo un diagrama Gantt con una estimación de la duración de cada tarea y dando un plazo de entrega aproximado de tres meses.

⁶PostgreSQL - <https://www.postgresql.org/>

⁷ClamAV - <https://www.clamav.net/>

⁸YARA - <https://virustotal.github.io/yara/>

⁹yara-python - <https://yara.readthedocs.io/>

¹⁰oletools - <https://github.com/decalage2/oletools>

¹¹Apache Tika - <https://tika.apache.org/>

¹²Mutagen - <https://mutagen.readthedocs.io/>

¹³Pillow - <https://python-pillow.org/>

¹⁴NumPy - <https://numpy.org/>

¹⁵Matplotlib - <https://matplotlib.org/>

¹⁶Requests - <https://requests.readthedocs.io/>

¹⁷Docker - <https://www.docker.com/>

¹⁸Gunicorn - <https://gunicorn.org/>

En la siguiente figura 3.1 se puede observar el diagrama Gantt inicial.

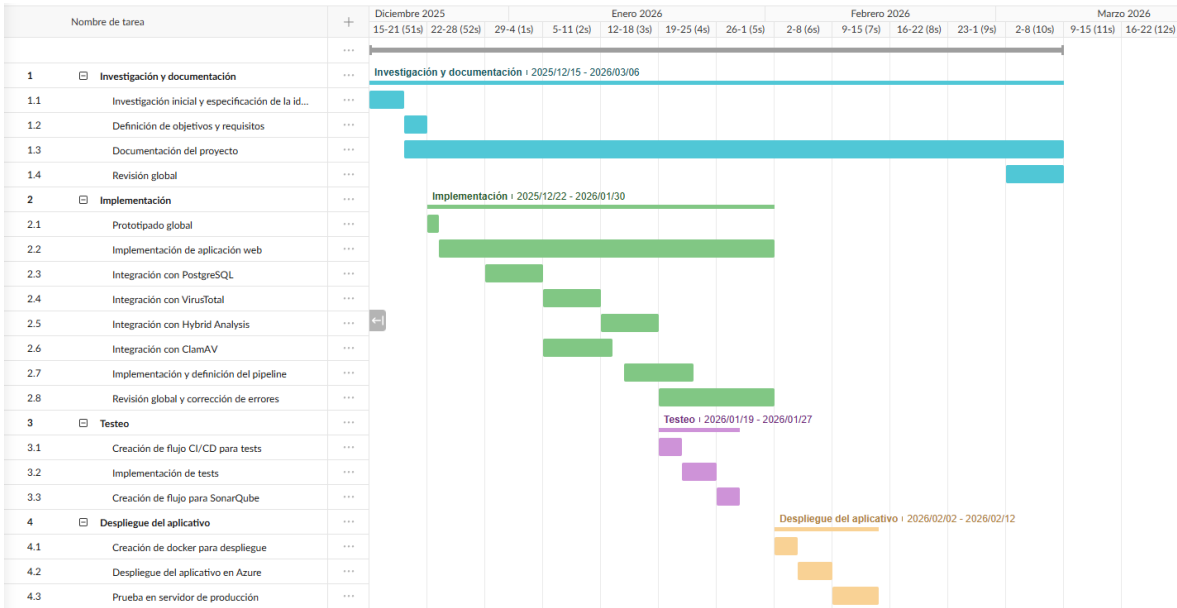


Figura 3.1: Diagrama gantt - estimación de duración de las tareas

Finalmente, debido a diferentes circunstancias mientras se desarrollaba el proyecto, la planificación inicial varió, haciendo que algunas tareas se retrasasen y otras se adelantasen.

En la siguiente figura 3.2 se puede observar el diagrama Gantt actualizado.

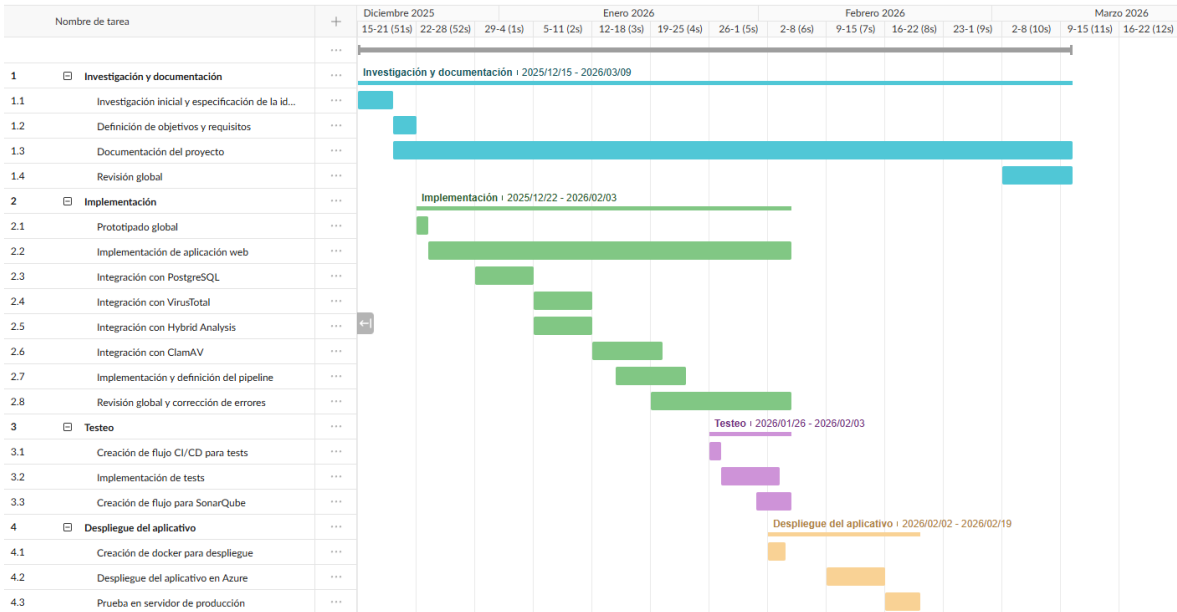


Figura 3.2: Diagrama gantt - duración real de las tareas

3.4. Organización

Este proyecto y documentación han sido desarrollado por el alumno Francisco Javier Molina Rojas, bajo supervisión del Dr. Juan Boubeta Puig y del Dr. Jesús Rosa Bilbao.

Durante la realización de las tareas del proyecto, se ha asumido un amplio conjunto de roles relacionados con la ingeniería de software; desde la recogida de requisitos funcionales y no funcionales, hasta la realización de tests.

Durante la fase de prototipado se ha utilizado la herramienta Figma¹⁹ para especificar el diseño para las diferentes pantallas del aplicativo. En la fase de implementación para codificar se ha utilizado el IDE Visual Studio Code²⁰ y para el control de versiones se ha creado un repositorio en Github²¹.

Por último, para la fase de pruebas se ha utilizado el *framework* Pytest²² junto con el cliente de pruebas que ofrece Flask.

3.5. Costes

Para visualizar los costes asociados a este proyecto, exponemos los datos en la siguiente tabla.

Concepto	Coste
Técnico Investigador Graduado	8572.78 €
Costes indirectos (10 %)	857 €
Licencias software	0 €
Total	9429.78 €

Tabla 3.1 - Desglose de costes

Especificando en cada coste:

- Técnico Investigador Graduado: coste fundamentado en la normativa presupuestaria de la UCA (Universidad de Cádiz) visible en el Capítulo VI (UCA, 2025) por 3 meses.
- Costes indirectos: contiene costes derivados personales y de proyecto.
- Licencias de software: la mayoría de las tecnologías utilizadas en este proyecto son de código abierto, aquellas que no lo son, tenían una versión gratuita que se ha utilizado; por lo que no se han incurrido en costes de licencias.

¹⁹Figma - <https://www.figma.com/>

²⁰Visual Studio Code - <https://code.visualstudio.com/>

²¹Repositorio de Github - <https://github.com/Javivi-MR/ForenAlyze>

²²pytest - <https://docs.pytest.org/>

3.6. Riesgos

En la próxima tabla se muestra un análisis de riesgos asociado a este proyecto, con su probabilidad, impacto y plan de mitigación.

Riesgo	Probabilidad	Impacto
Fuga de datos o vulnerabilidades de seguridad	Baja	Alto
Herramientas externas no disponibles	Muy baja	Alto
No cumplir del plazo de entrega	Medio	Medio

Tabla 3.2 - Tabla de riesgos

Estos riesgos implican:

- Fuga de datos o vulnerabilidades de seguridad: dado que el aplicativo maneja información, existe el riesgo de que se produzcan fugas de datos.
- Herramientas externas no disponibles: dado que el aplicativo depende de herramientas externas, existe el riesgo de que estas no estén disponibles en algún momento.
- No cumplir del plazo de entrega: dado que el proyecto tiene un plazo de entrega, existe el riesgo de no cumplirlo.

El Plan de mitigación a seguir es el siguiente:

- Fuga de datos o vulnerabilidades de seguridad: se mantendrá el software actualizado, se aplicarán buenas prácticas de seguridad en el desarrollo y se realizarán pruebas de seguridad con regularidad.
- Herramientas externas no disponibles: se diseñará el aplicativo de forma modular para que pueda funcionar con herramientas alternativas en caso de que alguna de las herramientas externas deje de estar disponible.
- No cumplir del plazo de entrega: se realizará un seguimiento regular del progreso del proyecto y se ajustará la planificación si es necesario.

3.7. Gestión de la configuración

Como se ha mencionado, el control de versiones se realiza mediante la plataforma GitHub, donde se ha creado un repositorio público para alojar el código fuente. El principal propósito es facilitar la descarga del software.

3.8. Aseguramiento de calidad

Con el fin de asegurar la calidad del software desarrollado, se han implementado dos medidas principalmente.

En primer lugar, se ha usado la extensión de SonarLint²³ para Visual Studio Code, que permite detectar y corregir problemas de calidad en el código durante el desarrollo.

En segundo lugar, se ha usado la herramienta de análisis estático de código SonarQube²⁴ para analizar el código fuente del proyecto y detectar posibles problemas de calidad, vulnerabilidades y errores.

²³SonarLint - <https://www.sonarlint.org/>

²⁴SonarQube - <https://www.sonarqube.org/>

Parte II

Desarrollo

4. Requisitos del Sistema

En este capítulo se describen los requisitos que debe cumplir el sistema. Estos abarcan los objetivos generales del sistema, los requisitos funcionales, los requisitos no funcionales y los requisitos de información.

4.1. Objetivos del Sistema

Los principales objetivos que el sistema debe cumplir son los siguientes:

- Integración de herramientas de análisis forense: el sistema debe integrar múltiples herramientas de análisis forense que permitan analizar distintos tipos de ficheros.
- Diseño modular de *pipeline*: el sistema debe permitir la creación de *pipelines* personalizados, permitiendo a los usuarios seleccionar las herramientas que desean usar.
- Permitir subida de distintos tipos de ficheros: el sistema debe permitir a los usuarios subir distintos tipos de ficheros, por ejemplo, imágenes, documentos, ejecutables...
- Permitir la trazabilidad de las acciones realizadas por los usuarios: el sistema debe registrar las acciones relevantes realizadas por los distintos usuarios.
- Ofrecer una interfaz web usable y funcional: el sistema debe ofrecer una interfaz web que facilite a los usuarios la interacción con el sistema.
- Exportación de informes: el sistema debe permitir a los usuarios exportar los resultados de los análisis en diferentes formatos.
- Ser sencillo de desplegar: el sistema debe ser fácil de desplegar en entornos de producción o desarrollo.

4.2. Requisitos funcionales

El sistema deberá realizar las siguientes funciones:

- Los administradores del sistema podrán crear nuevos usuarios, asignar administradores, gestionar el espacio de almacenamiento y eliminar usuarios.
- Los usuarios podrán subir ficheros con extensión exe, docx, xlsx, pdf, jpg, png y wav. El sistema deberá validar que los ficheros subidos tengan una extensión permitida y que no superen un tamaño máximo de 100 MB.
- El sistema asociará cada fichero subido a un usuario autenticado, garantizando la trazabilidad.

- El sistema calculará automáticamente los valores hash (MD5, SHA1 y SHA256) de cada fichero subido.
- El sistema ejecutará un *pipeline* de análisis modular que combine técnicas de análisis estático, dinámico y heurístico.
- El sistema permitirá la ejecución de reglas YARA que los usuarios podrán crear, modificar y eliminar.
- El sistema integrará un motor antivirus para la detección automática de *malware* mediante firmas conocidas.
- El sistema permitirá la integración opcional con servicios externos de análisis, como *sandboxes* o motores de reputación.
- El sistema extraerá automáticamente los metadatos relevantes de los ficheros subidos.
- El sistema detectará la presencia de macros o código embebido sospechoso en documentos ofimáticos.
- El sistema implementará mecanismos de detección de posibles técnicas de esteganografía en imágenes y archivos de audio.
- El sistema clasificará automáticamente los resultados del análisis y generará un informe con veredicto global asociado a cada fichero.
- El sistema generará alertas en función del nivel de severidad detectado.
- Los usuarios podrán consultar el historial de análisis realizados sobre sus ficheros.
- El sistema registrará las acciones relevantes realizadas por los usuarios con fines de auditoría y trazabilidad.
- El sistema permitirá la exportación de los resultados del análisis en diferentes formatos.
- El usuario podrá editar su perfil, incluyendo la contraseña y otros datos personales.
- El usuario podrá consultar los *logs* personales de su actividad y si es administrador, los *logs* de actividad de otros usuarios.
- El usuario podrá editar y gestionar sus reglas YARA personalizadas.

4.3. Requisitos no funcionales

El sistema deberá cumplir los siguientes requisitos:

- Usabilidad: La interfaz web deberá ser intuitiva, accesible, limpia y orientada a facilitar la interpretación de los resultados del análisis.
- Seguridad: El sistema deberá implementar mecanismos de autenticación y control de acceso basados en roles.

- Rendimiento: El sistema deberá ser capaz de procesar múltiples análisis concurrentes sin degradación significativa del rendimiento.
- Modularidad: El diseño deberá seguir un enfoque modular que facilite el mantenimiento, ampliación y sustitución de componentes.
- Trazabilidad: El sistema deberá registrar de forma persistente las acciones relevantes realizadas por los usuarios.
- Portabilidad: El sistema deberá poder desplegarse en diferentes entornos de desarrollo y producción con un proceso de configuración simplificado.
- Documentación: El proyecto deberá estar adecuadamente documentado.
- Código abierto: El sistema deberá publicarse en un repositorio público.

4.4. Requisitos de información

La información que debe gestionar el sistema se agrupa en las siguientes categorías:

- Usuarios:
 - Identificador único de usuario.
 - Nombre de usuario.
 - Contraseña (almacenada como *hash* seguro).
 - Rol (usuario estándar o administrador).
 - *Path* hacia la imagen de perfil.
 - Número de notificaciones pendientes.
 - Registros de actividad asociados.
 - Ficheros subidos por el usuario.
 - Análisis solicitados por el usuario.
 - Alertas asociadas al usuario.
- Ficheros subidos:
 - Identificador único de fichero.
 - Usuario propietario del fichero.
 - Nombre original del fichero.
 - Nombre interno almacenado.
 - Ruta física de almacenamiento.
 - Fecha y hora de subida.
 - Tamaño en bytes.

- Tipo de fichero.
- Tipo MIME detectado.
- Hashes principales: MD5, SHA1 y SHA256.
- Análisis realizados sobre dicho fichero.
- Alertas generadas asociadas al fichero.
- Análisis realizados:
 - Identificador único de análisis.
 - Fichero analizado.
 - Usuario que solicita o al que se asocia el análisis.
 - Fecha y hora de realización del análisis.
 - Hashes del fichero en el momento del análisis.
 - Tipo MIME utilizado en el análisis.
 - Resultados de motores y reglas:
 - Resultado de reglas YARA.
 - Resultado del análisis antivirus local.
 - Resultado de consultas a VirusTotal.
 - Detecciones específicas:
 - Presencia de macros sospechosas en documentos Office.
 - Indicadores de esteganografía o contenido oculto.
 - Información de análisis de audio.
 - Resultado de integración con *sandbox* dinámica:
 - Puntuación de riesgo o *sandbox score*.
 - Información resumida del análisis dinámico.
 - Veredicto global del análisis.
 - Resumen textual de los hallazgos principales.
 - Información de versión:
 - Versión del motor de análisis.
 - Versión del conjunto de reglas utilizado.
 - Alertas generadas a partir de este análisis.
- Alertas:
 - Identificador único de alerta.
 - Usuario destinatario de la alerta.

- Fichero asociado a la alerta.
- Análisis asociado a la alerta.
- Título de la alerta.
- Severidad.
- Descripción detallada de la alerta.
- Indicador de si la alerta ha sido leída o no.
- Fecha y hora de creación de la alerta.
- Registros de actividad:
 - Identificador único de registro.
 - Fecha y hora del evento.
 - Usuario asociado al evento.
 - Nombre de usuario en el momento del evento.
 - Dirección IP de origen.
 - *User-Agent* del cliente.
 - Acción realizad.
 - Recurso o módulo afectado.
 - Estado del evento.
 - Mensaje legible que describe el evento.
 - Detalles adicionales en formato estructurado.

5. Análisis del Sistema

En este capítulo se realiza un análisis detallado del sistema, especificando los diferentes modelos; como el conceptual, los casos de uso, los de interfaz de usuario y los de comportamiento.

5.1. Modelo Conceptual

Según los requisitos de información que hemos definido en la sección 4.4, hemos realizado el siguiente modelo conceptual de datos, que muestra las entidades y relaciones principales del sistema y se representa en la figura 5.1.

A continuación se describen brevemente las principales clases del modelo conceptual:

- **Fichero:** representa cada fichero que es subido al sistema para su análisis. Para ello, se define un identificador único junto con atributos como el propietario, el nombre original y el nombre almacenado, la fecha de subida, la ruta a la imagen asociada, el tamaño, el tipo de fichero, el tipo MIME y los distintos *hashes* calculados. Estos datos permiten localizar el fichero, vincularlo a un usuario y garantizar su integridad.
- **Usuario:** recoge la información básica de las personas que utilizan la plataforma. Incluye un identificador, el nombre, la contraseña almacenada como *hash*, el rol y la ruta a su imagen de perfil. Esta clase permite gestionar la autenticación, la autorización y la personalización de la experiencia de uso.
- **Análisis:** modela cada ejecución del *pipeline* de análisis sobre un fichero concreto. Contiene un identificador propio, la referencia al fichero y a su propietario, la fecha de análisis y los resultados obtenidos de los distintos módulos: reglas YARA, VirusTotal, antivirus, detección de macros, detección de esteganografía, puntuación de *sandbox*, así como un veredicto global y un resumen. De este modo se almacena de forma estructurada la evidencia generada por el sistema.
- **Alertas:** agrupa las notificaciones que se generan a partir de un análisis. Incluye el identificador, el usuario destinatario, el fichero y el análisis asociados, junto con el título, la severidad, la descripción, el indicador de lectura y la fecha de creación. Esta clase permite priorizar y comunicar al usuario aquellos resultados que requieren especial atención.
- **Log:** registra los eventos relevantes que se producen en la aplicación. Para cada entrada se guarda un identificador, el usuario implicado, la dirección *IP*, el *user agent*, la acción realizada, el estado, un mensaje y los detalles asociados. Gracias a esta clase se facilita la trazabilidad de las operaciones de la actividad del sistema.

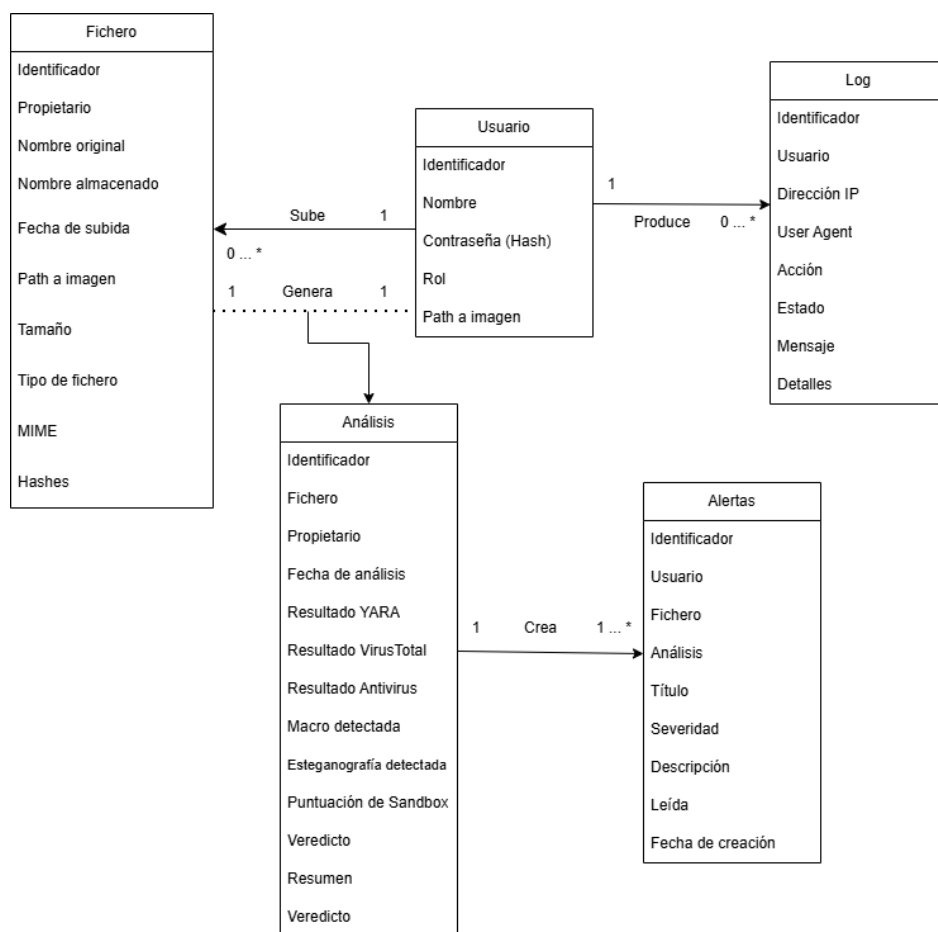


Figura 5.1: Diagrama conceptual de datos

5.2. Modelo de Casos de Uso

Según los requisitos funcionales descritos en la sección 4.2, hemos definido los siguientes casos de uso principales para el sistema.

Nombre del CU:	Subida fichero
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, puede subir un fichero para su análisis.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado y hay espacio disponible tanto por parte del sistema como del usuario.
Postcondiciones	1. El sistema almacena el fichero subido y genera un informe asociado a este
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, hace clic en “Dashboard”. 2. Posteriormente, hace clic en “Upload file”. 3. El usuario arrastra y suelta en el cuadro central el fichero que desea analizar. 4. El usuario hace clic en “Upload and analyse”. 5. El sistema habrá almacenado el fichero y lo habrá analizado, creando un informe con la información detallada.
Flujo alternativo	3a. En el paso 3, el usuario hace clic en “Choose file”, y le aparece un menú del SO donde podrá seleccionar el fichero a analizar y continuar con el caso de uso.
Excepciones	3b. En el paso 3, el usuario introduce un fichero que supera el tamaño límite de 100 MB. 4. El usuario hace clic en “Upload and analyse”. 5. El sistema muestra un mensaje de error indicando que ese fichero excede el límite de tamaño. 3c. En el paso 3, el usuario no selecciona ningún fichero. 4. El usuario hace clic en “Upload and analyse”. 5. El sistema no avanzará hasta que el usuario seleccione un fichero.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema. 2. El usuario debe tener espacio suficiente. 3. El sistema debe tener espacio suficiente.

Figura 5.2: Caso de uso Subir fichero

Nombre del CU:	Crear usuario
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado y es administrador, puede crear un nuevo usuario con o sin privilegios.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El usuario debe tener privilegios de administrador. 3. El sistema está iniciado y hay espacio disponible.
Postcondiciones	1. El sistema crea un nuevo usuario con los datos especificados por el usuario.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, sitúa el ratón encima de su perfil, y selecciona la opción "User Management". 2. A la izquierda el nombre de usuario a crear y su contraseña. 3. Selecciona si quiere o no que tenga privilegios de administrador. 4. Pulsa en "Create user". 5. El sistema habrá creado al usuario especificado.
Flujo alternativo	-
Excepciones	2a. En el paso 2, el usuario introduce un nombre de usuario que ya existe en el sistema. 3. Selecciona si quiere o no que tenga privilegios de administrador. 4. Pulsa en "Create user". 5. El sistema muestra un mensaje de error indicando que ese nombre de usuario ya existe en el sistema.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema. 2. El usuario debe tener permisos de administrador. 3. El sistema debe tener espacio suficiente.

Figura 5.3: Caso de uso Crear usuario

Nombre del CU:	Cambiar contraseña
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, este puede cambiar su contraseña.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado.
Postcondiciones	1. El sistema cambia la contraseña del usuario.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, sitúa el ratón encima de su perfil, y selecciona la opción “Edit profile” 2. Posteriormente, hace clic en “Change password”. 3. En el modal que aparece, el usuario introduce su anterior contraseña, y la nueva contraseña. 4. El sistema realiza el cambio y lo confirma al usuario.
Flujo alternativo	-
Excepciones	3a. El usuario no introduce correctamente la contraseña anterior. 4. El sistema informa al usuario de que la contraseña introducida no es correcta.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema.

Figura 5.4: Caso de uso Cambiar contraseña

Nombre del CU:	Cambiar foto de perfil
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, este puede cambiar su foto de perfil.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado y tiene espacio suficiente.
Postcondiciones	1. El sistema cambia la contraseña del usuario.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, sitúa el ratón encima de su perfil, y selecciona la opción “Edit profile” 2. Posteriormente, hace clic en “Change profile photo”. 3. En el modal que aparece, el usuario introduce la foto que desea tener. 4. El sistema realiza el cambio y lo confirma al usuario.
Flujo alternativo	-
Excepciones	3a. El usuario no introduce ninguna imagen. 4. El sistema informa al usuario de que no ha seleccionado ninguna imagen y no realiza ningún cambio.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema. 2. El usuario debe haber subido al menos un fichero. 3. El sistema debe tener espacio suficiente.

Figura 5.5: Caso de uso Editar foto de perfil de usuario

Nombre del CU:	Editar regla YARA
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, si es administrador, puede editar reglas YARA.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado. 3. El usuario debe ser administrador. 4. El sistema debe tener al menos una regla YARA almacenada.
Postcondiciones	1. El sistema edita la regla YARA.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, hace clic en “Dashboards” y posteriormente hace clic en “YARA rules” 2. El usuario selecciona la regla a editar y hace clic en “edit”. 3. El usuario hace los cambios sintácticos que considere necesario y hace clic en “Save changes”. 4. El sistema valida el fichero y lo edita.
Flujo alternativo	-
Excepciones	3a. El usuario edita con errores de sintaxis la regla. 4. El sistema no acepta los cambios y notifica al usuario.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema. 2. El usuario debe ser administrador. 3. El sistema debe tener al menos una regla YARA almacenada.

Figura 5.6: Caso de uso Editar regla YARA

Nombre del CU:	Consultar logs
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, este puede consultar sus logs y si es administrador, los logs del resto de usuarios.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado.
Postcondiciones	1. El sistema muestra los logs correspondientes.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, hace clic en “Dashboards” y posteriormente hace clic en “Logs”. 2. El sistema muestra los logs correspondientes al usuario y en caso de tener administrador, mostraría los logs correspondientes a todos los usuarios.
Flujo alternativo	-
Excepciones	-
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema.

Figura 5.7: Caso de uso Consultar *logs* de actividad del sistema

Nombre del CU:	Subir regla YARA
Creado por:	Francisco Javier Molina Rojas
Fecha	05/02/2026
Descripción	Cuando un usuario está registrado en el sistema, si es administrador, puede subir reglas YARA.
Actores	Usuario y sistema.
Precondiciones	1. El usuario debe estar registrado en el sistema. 2. El sistema está iniciado. 3. El usuario debe ser administrador. 4. El sistema debe tener espacio disponible.
Postcondiciones	1. El sistema crea una nueva regla YARA.
Flujo	1. El usuario, una vez ha iniciado sesión en el sistema, hace clic en “Dashboards” y posteriormente hace clic en “YARA rules” 2. El usuario hace clic en “upload new rule file”. 3. El usuario selecciona un fichero con la regla YARA y con extensión “.yar”. 4. El sistema valida el fichero, lo crea, y lo añade a la lista de reglas YARA del sistema.
Flujo alternativo	-
Excepciones	3a. El usuario selecciona un fichero que no es .yar o no tiene la sintaxis correcta de reglas yara. 4. El sistema no acepta el fichero y notifica al usuario.
Requisitos	Para poder realizar este caso de uso se tiene que cumplir las siguientes condiciones: 1. El usuario debe estar registrado en el sistema. 2. El usuario debe ser administrador. 3.

Figura 5.8: Caso de uso Subir nueva regla YARA

5.3. Modelo de Interfaz de Usuario

El usuario puede interactuar con el sistema a través de diferentes páginas web. En el diagrama de navegación de la figura 5.9 se exponen las diferentes páginas y cómo el usuario puede navegar entre ellas.

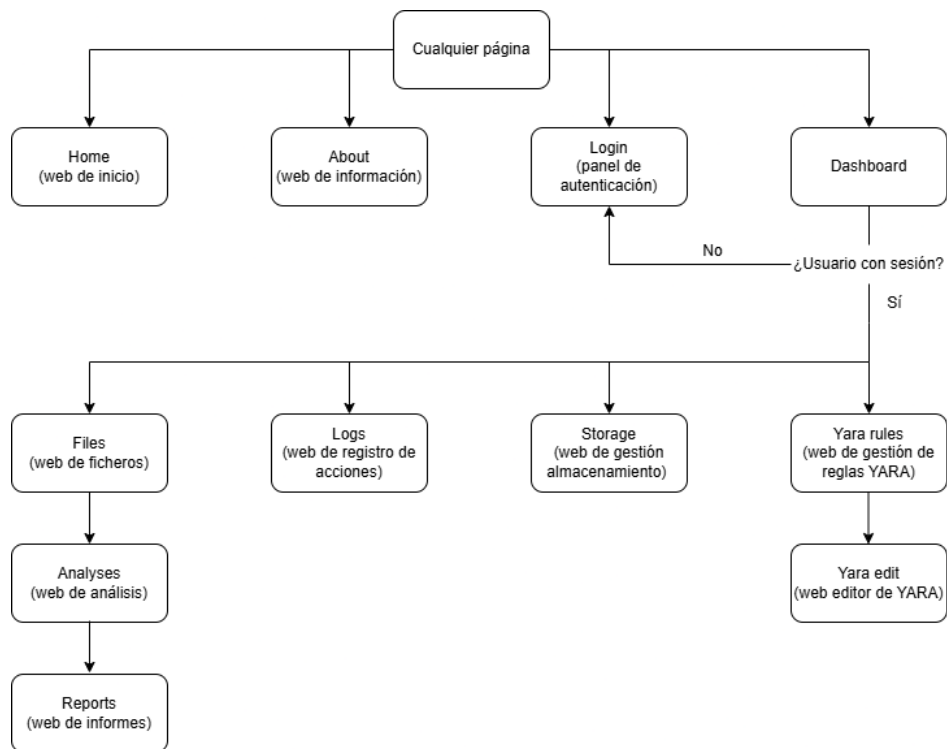


Figura 5.9: Diagrama de navegación

5.4. Modelo de Comportamiento

Según los casos de uso especificados en la sección 5.2, hemos definido los diagramas de secuencia de operaciones que ilustran el comportamiento del sistema.

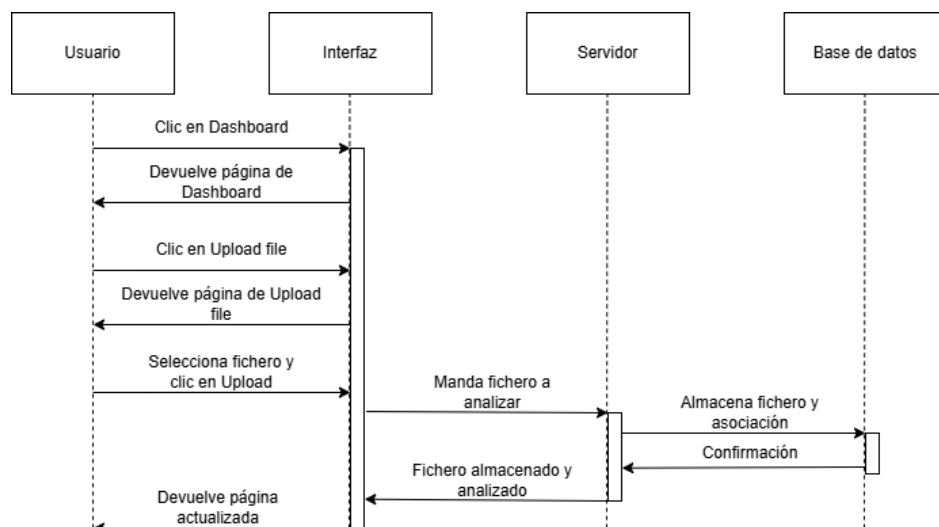


Figura 5.10: Diagrama de secuencia Subir fichero

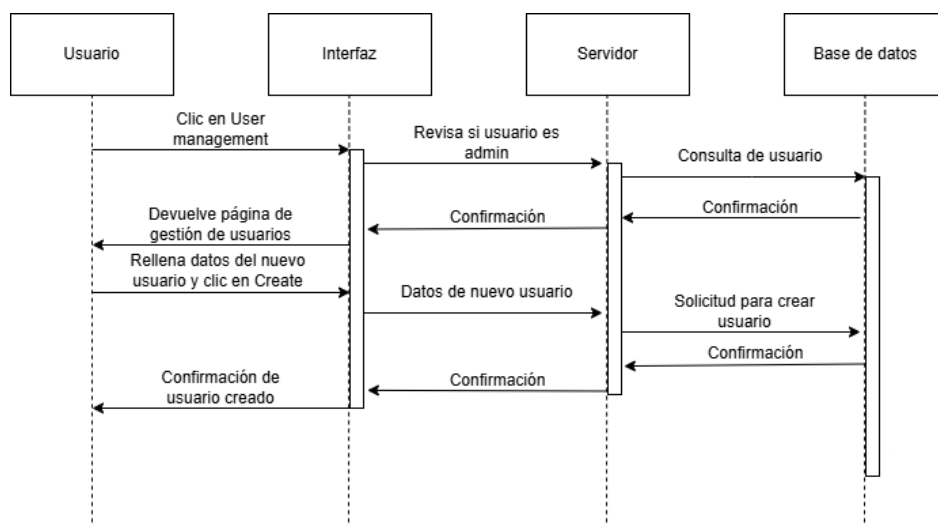


Figura 5.11: Diagrama de secuencia Crear usuario

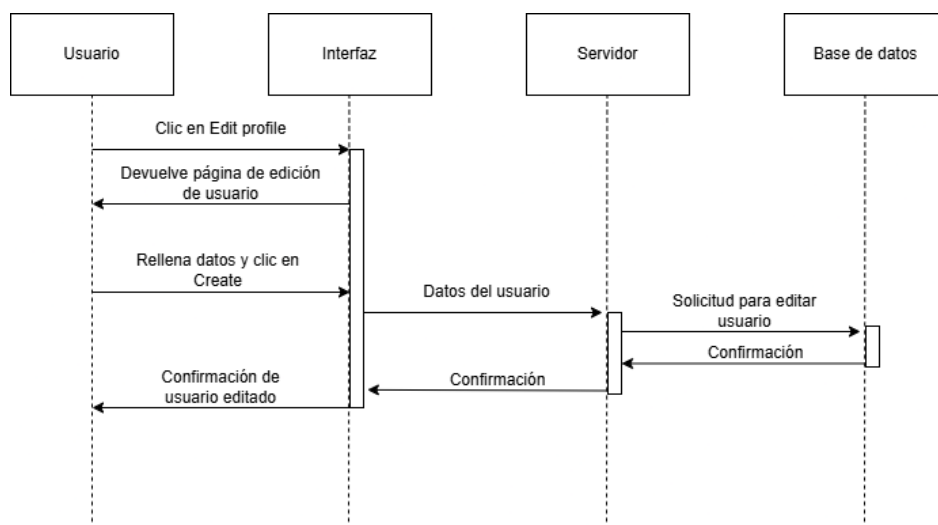


Figura 5.12: Diagrama de secuencia Cambiar contraseña

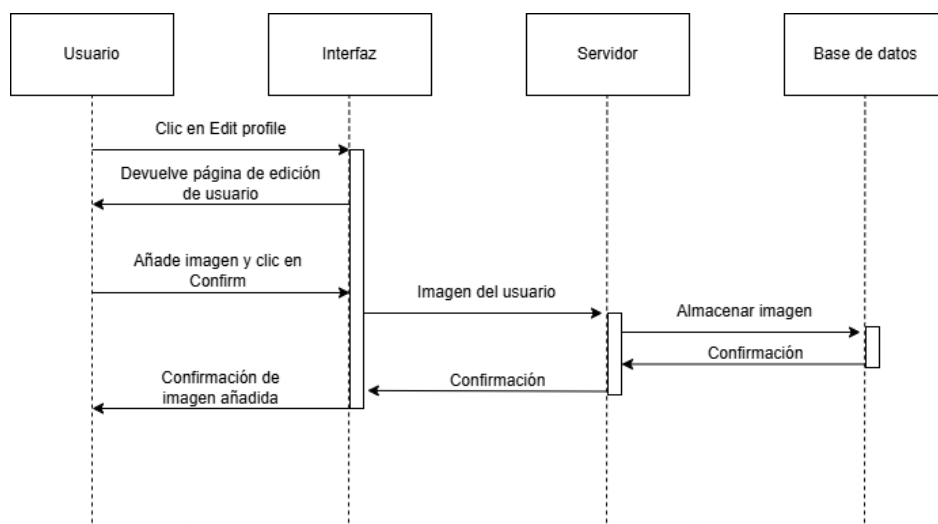


Figura 5.13: Diagrama de secuencia Editar foto de perfil de usuario

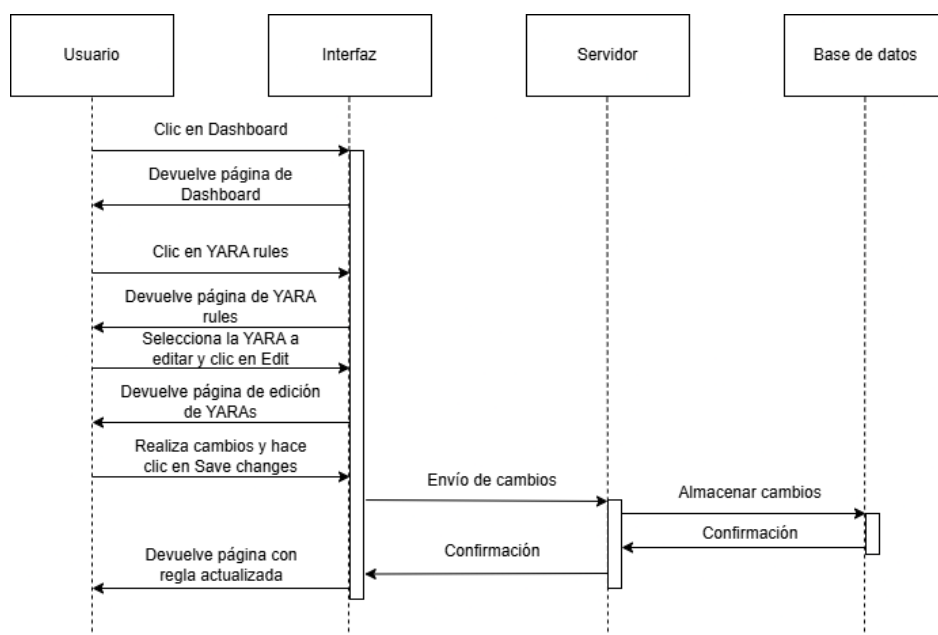


Figura 5.14: Diagrama de secuencia Editar regla YARA

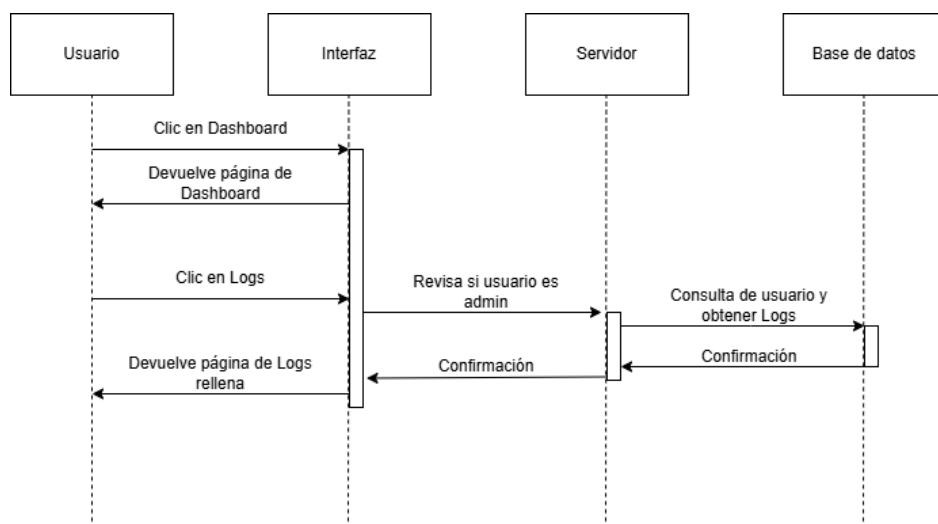


Figura 5.15: Diagrama de secuencia Consultar *logs* de actividad

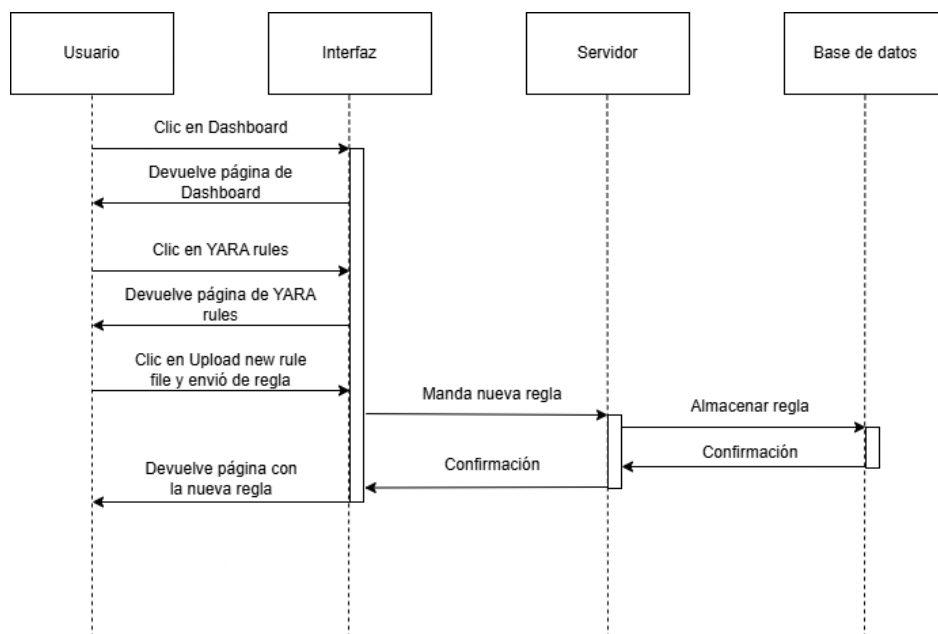


Figura 5.16: Diagrama de secuencia Subir nueva regla YARA

6. Diseño del Sistema

En este capítulo se describe el diseño del sistema; incluyendo la arquitectura del sistema, parametrización del software, diseño físico de datos y el diseño de la interfaz de usuario.

6.1. Arquitectura del Sistema

La arquitectura del sistema está dividida en diferentes directorios y ficheros. En el directorio raíz encontramos el script principal *run.py*, que es el encargado de inicializar el servidor y ponerlo a disposición de los usuarios.

Las principales tecnologías usadas para el desarrollo del sistema son las siguientes:

- Python: lenguaje de programación usado para el desarrollo del sistema.
- Flask: *framework* de Python usado para el desarrollo del aplicativo web.
- Jinja2: motor de plantillas empleado para generar los ficheros HTML a partir de los datos recibidos por el servidor.
- Bootstrap 5 y AOS: librerías de UI que crean una interfaz responsiva y con animaciones.
- Flask-SQLAlchemy y PostgreSQL: SGBD relacional utilizado para almacenar usuarios, ficheros, análisis, alertas y registros de actividad.
- Flask-Login: librería encargada de la autenticación de usuarios y la gestión de sesiones seguras.
- ClamAV y reglas YARA: herramientas de análisis de ficheros basadas en firmas y reglas integradas en el *pipeline* para detectar patrones maliciosos.
- Apache Tika: servicio externo usado para extraer texto y metadatos de documentos Office y PDF.
- Docker y Docker Compose: tecnologías de contenedorización empleadas para desplegar la aplicación, la base de datos y servicios auxiliares como Tika.

6.1.1. Diseño de alto nivel

El sistema se ha diseñado con una arquitectura modular y escalable, siguiendo el patrón *Layers*, que separa claramente las responsabilidades en diferentes capas y módulos. Esto facilita el mantenimiento, la extensibilidad y la reutilización de componentes.

Como se puede apreciar en el diagrama de contexto de la Figura 6.1, al sistema lo componen personas y tres sistemas distintos: la capa de interfaz, la capa de servidor y la capa de base de datos.

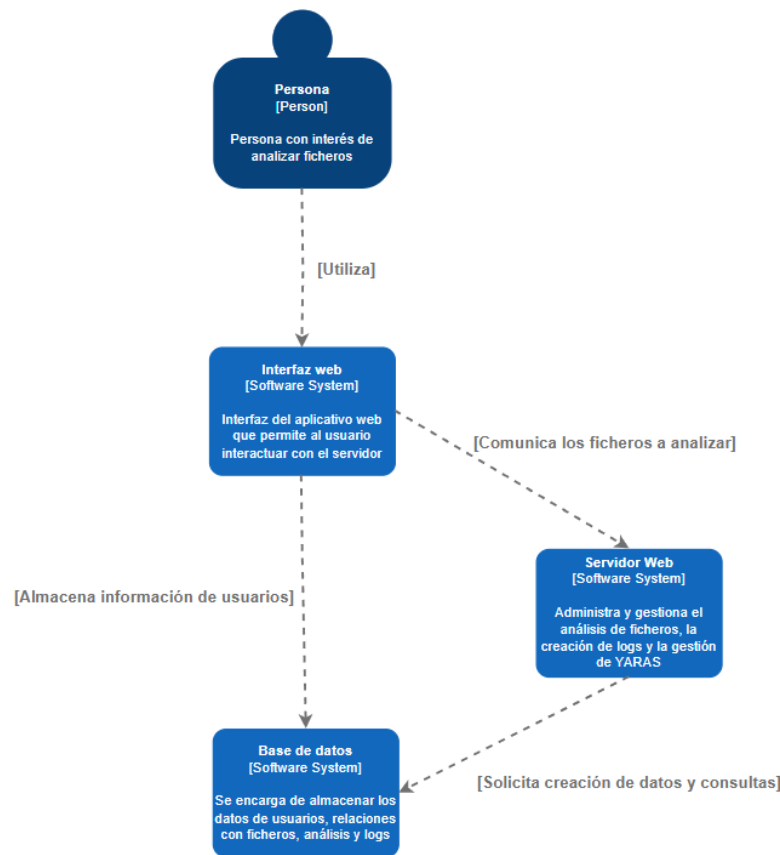


Figura 6.1: Diagrama de contexto

Primero vamos a analizar la capa de interfaz (véase Figura 6.2). Esta es la encargada de gestionar la interacción con el usuario y se compone de los siguientes módulos:

- Servidor WSGI: es el servidor que se encarga de recibir y procesar las peticiones HTTP.
- Aplicación de Flask: es el módulo principal del sistema, que se encarga de gestionar las rutas y las vistas. Su diagrama de contexto se muestra en la figura 6.3.
- Gestor de sesiones: se encarga de gestionar la autenticación y las sesiones de los usuarios. Su diagrama de contexto se muestra en la figura 6.4.

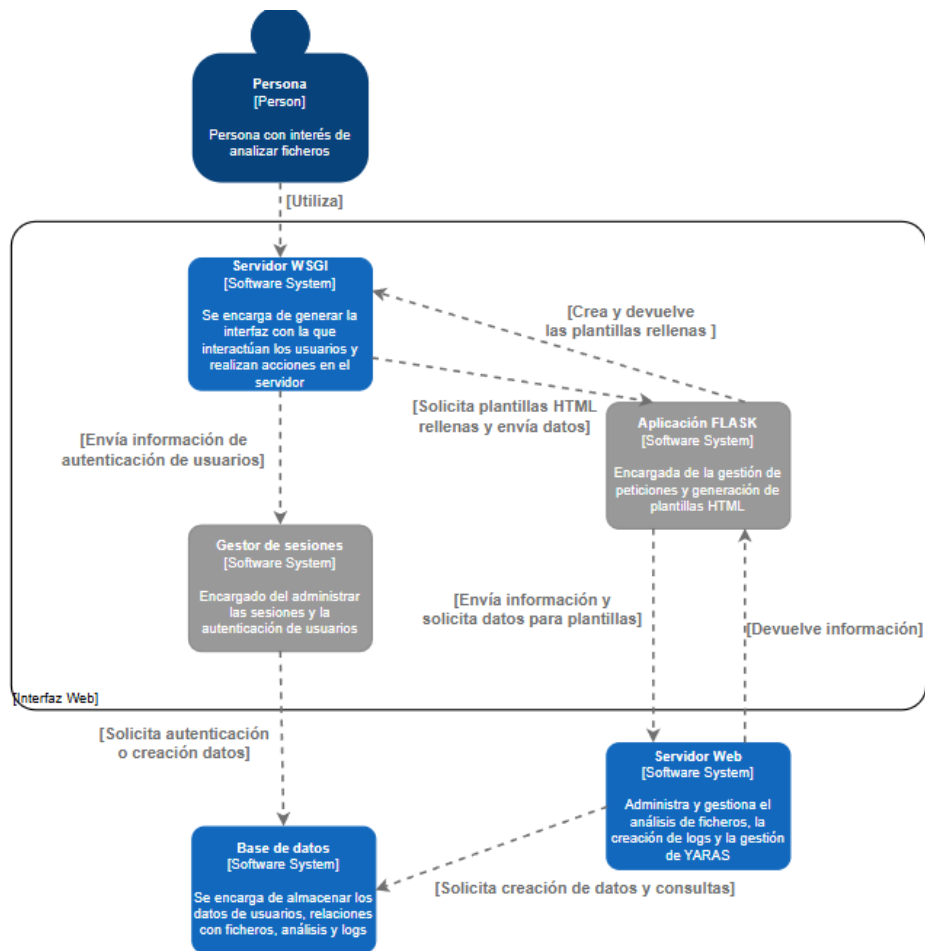


Figura 6.2: Diagrama de contexto - Interfaz

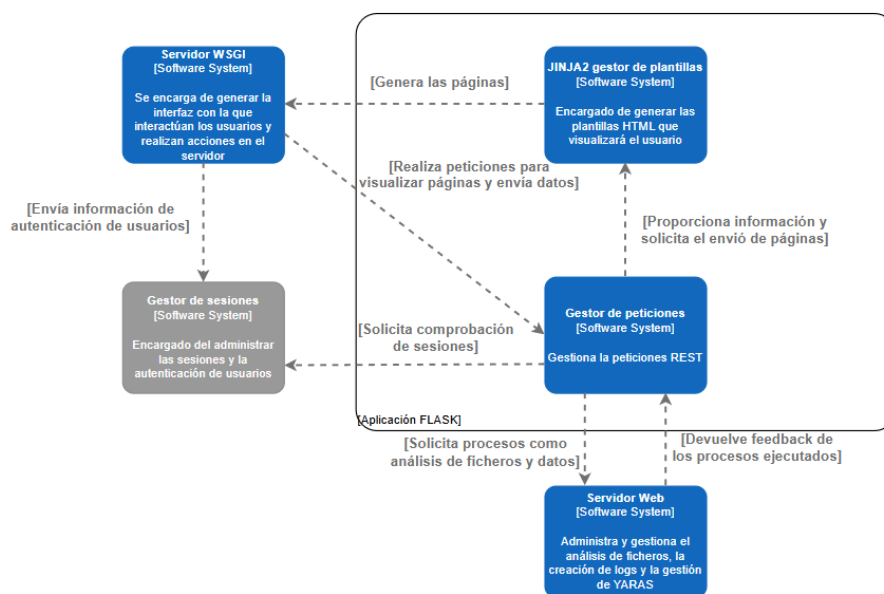


Figura 6.3: Diagrama de contexto - Aplicación Flask

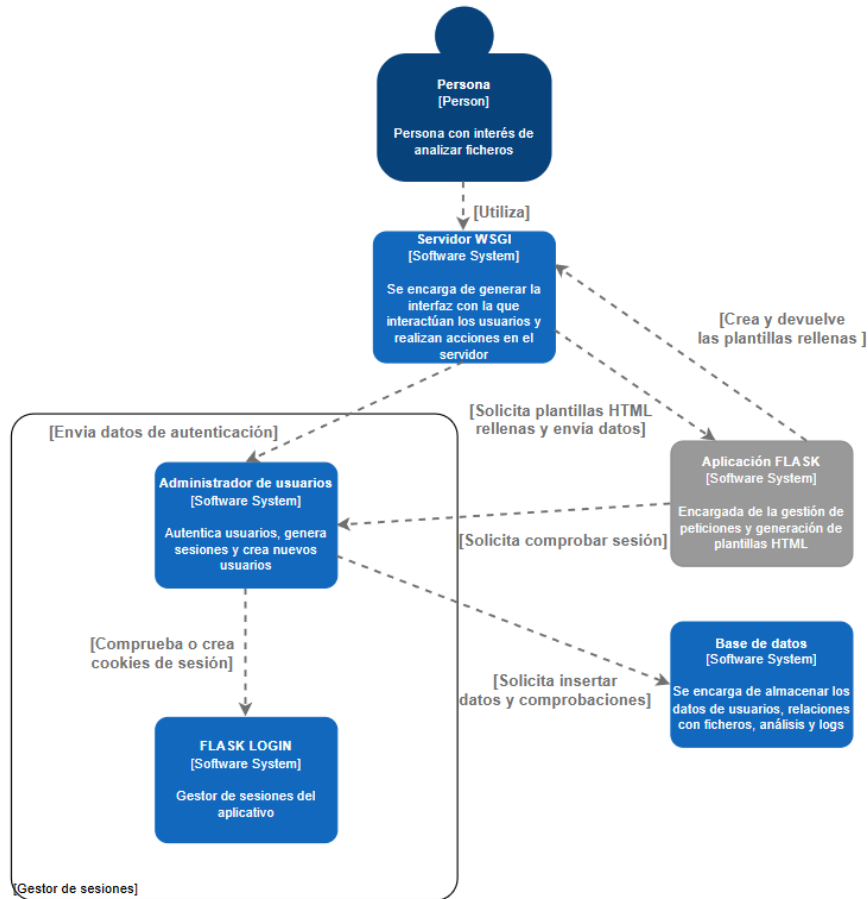


Figura 6.4: Diagrama de contexto - Gestor de sesiones

La siguiente capa es la capa de servidor (véase Figura 6.5). Esta capa es la encargada de la lógica de negocio del sistema, estando compuesta por los siguientes módulos:

- Gestor de procesos: gestiona los diferentes procesos que puede solicitar el usuario.
- *Pipeline* de análisis: se encarga de dirigir todos los procesos relacionados con el análisis de los ficheros subidos por los usuarios. Su diagrama de contexto se muestra en la figura 6.6.
- Gestor de datos de usuarios y reglas YARA: se encarga de la creación, edición y eliminación de datos de usuarios y reglas YARA.
- Gestor de *logs*: se encarga de la creación de registros de actividad creados por la interacción de los usuarios con el sistema.

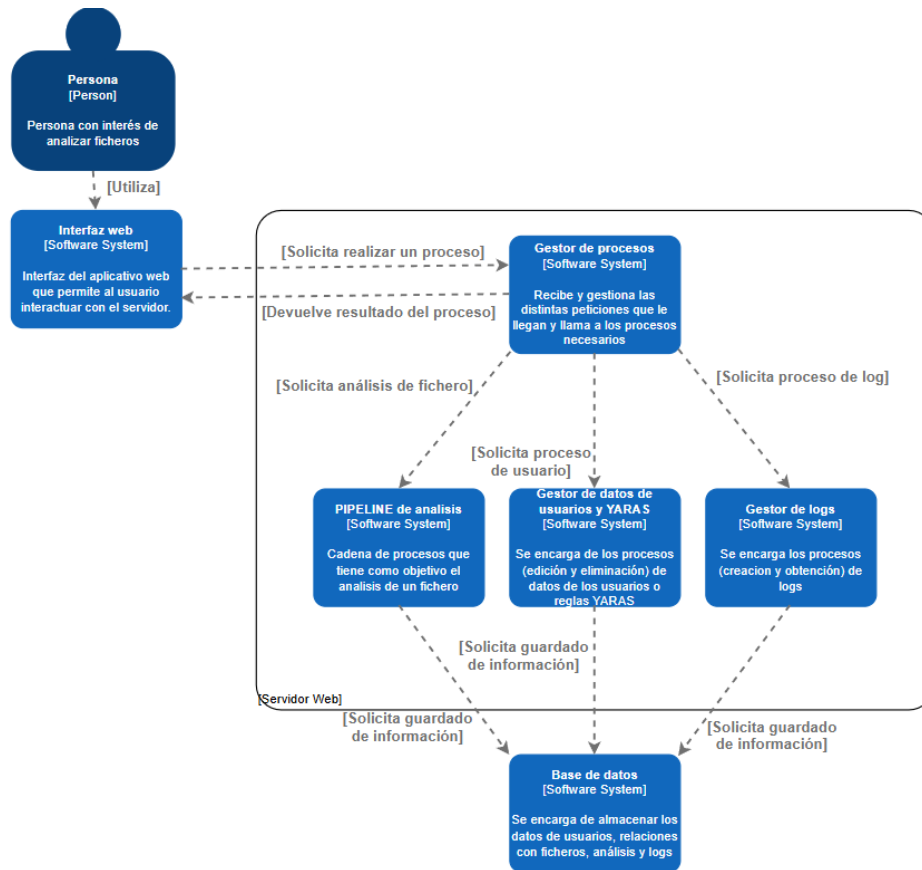


Figura 6.5: Diagrama de contexto - Servidor

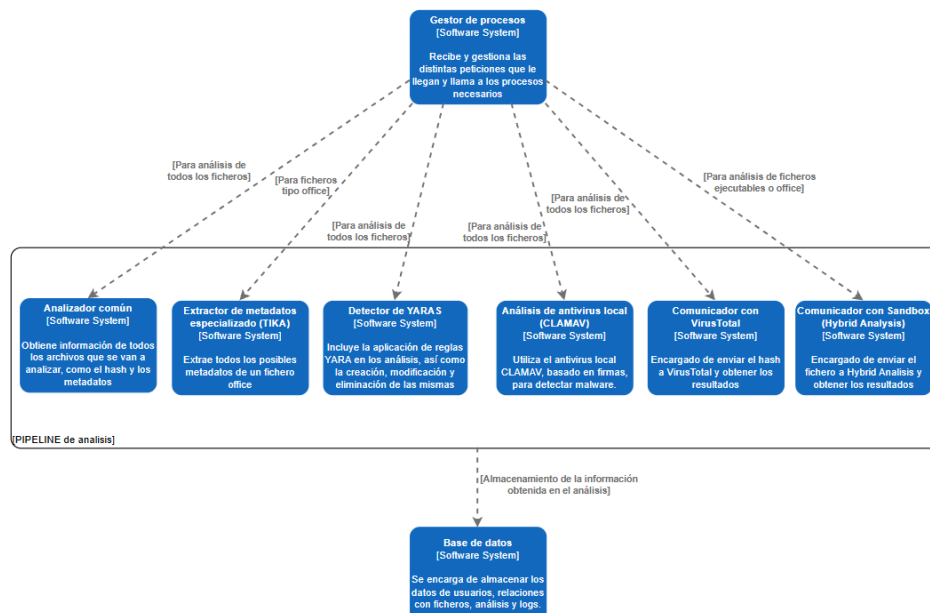


Figura 6.6: Diagrama de contexto - Pipeline de análisis

La última capa es la capa de base de datos (véase Figura 6.7). Esta capa es la encargada de almacenar toda la información del sistema y se compone de los siguientes

módulos:

- Base de datos PostgreSQL: SGBD relacional usado para almacenar la información de los usuarios y las relaciones que tienen con los ficheros subidos, los análisis realizados y los registros de actividad.
- Sistema de ficheros de SO: se encarga de almacenar los ficheros subidos por los usuarios y las reglas YARA.

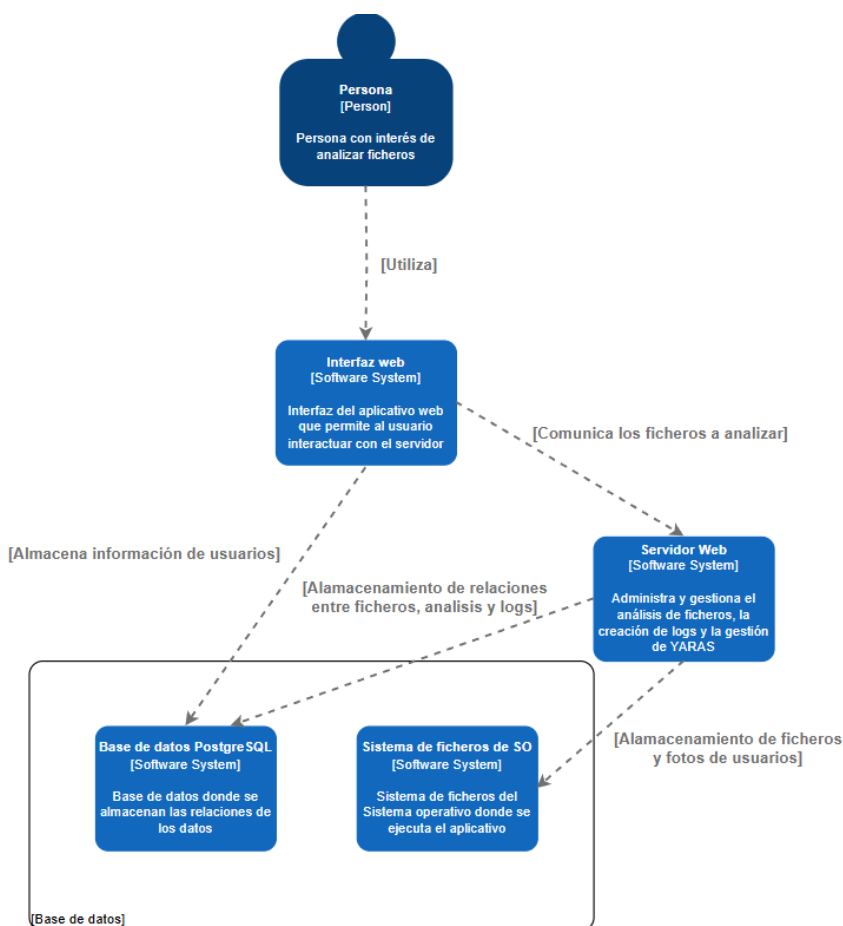


Figura 6.7: Diagrama de contexto - Base de datos

6.1.2. Diseño detallado

En este apartado se describirán en detalle los diferentes módulos que componen el sistema, sus funcionalidades, sus características y su interacción con el resto de módulos.

Análisis de ficheros:

El análisis de ficheros es una de las funcionalidades principales del sistema. Este proceso se encarga de analizar los ficheros subidos por los usuarios, siguiendo un *pipeline* de análisis.

Este *pipeline* es modular y escalable, es decir, está programado de tal forma que es fácil añadir nuevos módulos de análisis sin necesidad de modificar el resto del sistema.

Según el tipo de fichero, el *pipeline* ejecutará una serie de módulos de análisis. A continuación, se describen los módulos que incluye el software base.

Análisis con antivirus local (ClamAV):

Este módulo se encarga de analizar los ficheros subidos usando el antivirus de código abierto ClamAV. Este antivirus está basado en firmas, es decir, compara el contenido del fichero con una base de datos de firmas de *malware* conocidas y si encuentra una coincidencia, el fichero es marcado como malicioso.

Se ha decidido usar este antivirus porque es de código abierto, es fácil de integrar en el sistema y ofrece una buena tasa de detección de *malware*.

Análisis con reglas YARA:

Este módulo se encarga de analizar los ficheros subidos usando reglas YARA. Las reglas YARA son un lenguaje de reglas para identificar *malware*. Estas reglas se basan en patrones de texto, secuencias de bytes o expresiones regulares.

Para la integración de este módulo se ha decidido usar la librería YARA-Python, que es una librería de Python que permite compilar y ejecutar reglas YARA desde el código Python.

Se ha decidido usar este módulo porque las reglas YARA son una herramienta muy potente que se utiliza ampliamente en el ámbito de la ciberseguridad.

Además, se ha asociado al repositorio del proyecto un repositorio de reglas YARA mantenido por la comunidad, para que los usuarios partan de una base de reglas actualizada.

Análisis con Apache Tika:

Este módulo se encarga de recopilar la máxima información posible de los ficheros subidos usando Apache Tika. Esta es una herramienta de código abierto que se encarga de extraer texto y metadatos de documentos Office y PDF, por lo que este módulo solo se ejecutará para este tipo de ficheros.

Para la integración de este módulo se ha decidido levantar un servicio externo de Apache Tika usando Docker, y el módulo se encargará de enviar los ficheros a este servicio para que realice el análisis y devuelva la información extraída.

Se ha decidido usar este módulo porque que Apache Tika es una herramienta muy potente y permite extraer una gran cantidad de información de los ficheros.

Análisis con VirusTotal:

Este módulo se encarga de analizar los ficheros subidos usando el servicio de análisis de *malware* en la nube VirusTotal. Este servicio permite enviar ficheros o las firmas de estos, para que sean analizados por una gran cantidad de antivirus y herramientas de análisis de *malware*. Los análisis dan como resultado una puntuación de detección, que indica el número de antivirus que han detectado el fichero como malicioso.

Para la integración de este módulo se ha decidido usar la API gratuita de VirusTotal, que a pesar de tener limitaciones en cuanto al número de peticiones, es suficiente para el uso que le daremos en el sistema.

Se ha decidido usar este módulo porque que VirusTotal es un servicio muy conocido y potente.

Análisis con Sandbox (Hybrid Analysis):

Este módulo se encarga de realizar una ejecución controlada de los ficheros subidos en un entorno aislado (*sandbox*) para observar su comportamiento. Para esto, se ha decidido usar el servicio de Hybrid Analysis; que es un servicio gratuito que permite realizar esta funcionalidad en la nube.

Este módulo esta programado para poder integrar cualquier servicio de análisis en *sandbox* en la nube como Cuckoo Sandbox o Any.Run, facilitando el cambio de servicio en caso de que el servicio actual deje de ser gratuito o deje de ofrecer un buen servicio.

Se ha decidido utilizar este módulo porque el análisis en *sandbox* es una técnica de evaluación dinámica, que muestra el comportamiento real del archivo.

Análisis de esteganografía:

Este módulo se encarga de intentar detectar la presencia de técnicas básicas de esteganografía en los ficheros subidos, centrándose principalmente en imágenes y ficheros de audio (WAV).

El objetivo es identificar indicios de que el contenido visible del fichero pueda estar ocultando información adicional en sus bits menos significativos, o en bloques de datos embebidos.

Para ello, se han implementado técnicas de análisis que buscan patrones inusuales en la distribución de los bits como chi-cuadrado y prueba a extraer posibles mensajes ocultos usando técnicas como LSB.

6.2. Parametrización del software base

ForenAlyze permite ajustar numerosos parámetros de configuración a través de variables de entorno definidas en el fichero *.env*, que son leídas por el módulo de configuración *config.py* y por el entorno de ejecución.

El objetivo de esto es adaptar la aplicación a distintos escenarios sin modificar el código fuente.

Las principales opciones de configuración son:

- **SECRET_KEY**: clave secreta utilizada por Flask para firmar las cookies de sesión y otros elementos de seguridad.

- **DATABASE_URL**: cadena de conexión a la base de datos principal.
- **STORAGE_QUOTA_MB**: cuota máxima de almacenamiento por usuario, en megabytes.
- **CLAMAV_PATH**: ruta opcional al ejecutable `clamscan` de ClamAV. Si no se especifica, el sistema intentará localizarlo a través de la variable de entorno `PATH`.
- **VIRUSTOTAL_API_KEY**: clave de API usada para autenticar las peticiones al servicio VirusTotal.
- **VIRUSTOTAL_ENABLED**: parámetro booleano (*true/false*) que permite habilitar o deshabilitar el módulo.
- **VIRUSTOTAL_CACHE_TTL_SECONDS**: tiempo de vida, en segundos, de la caché en memoria utilizada para almacenar respuestas de VirusTotal y evitar peticiones repetidas para los mismos hashes.
- **YARA_ENABLED**: parámetro booleano (*true/false*) que activa o desactiva el escaneo de ficheros mediante reglas YARA.
- **YARA_RULES_PATH**: ruta al directorio que contiene las reglas YARA.
- **SANDBOX_ENABLED**: parámetro booleano (*true/false*) que controla si se activa el módulo de análisis dinámico en *sandbox*.
- **SANDBOX_MODE**: modo de funcionamiento del módulo de *sandbox*. Puede tomar valores como `disabled`, `file` o `hybrid_analysis`.
- **SANDBOX MOCK_RESULT_PATH**: ruta opcional a un fichero JSON con resultados de *sandbox* previamente generados.
- **HYBRID_ANALYSIS_API_KEY**: clave de API para integrar ForenAlyze con Hybrid Analysis cuando se usa el modo `hybrid_analysis`.
- **HYBRID_ANALYSIS_API_URL**: URL base de la API de Hybrid Analysis.
- **HYBRID_ANALYSIS_PUBLIC_URL**: URL pública del portal web de Hybrid Analysis.
- **HYBRID_ANALYSIS_ENV_ID**: identificador numérico del entorno de ejecución configurado en Hybrid Analysis.
- **TIKA_ENABLED**: parámetro booleano (*true/false*) que indica si el módulo de análisis con Apache Tika está activo.
- **TIKA_SERVER_URL**: dirección del servidor Tika accesible vía HTTP.
- **TIKA_TIMEOUT_SECONDS**: tiempo máximo de espera, en segundos, para las peticiones al servidor Tika antes de considerar que han fallado.
- **TIKA_MAX_TEXT_CHARS**: número máximo de caracteres de texto extraído que se almacenarán por documento, con el fin de evitar que la base de datos crezca de forma descontrolada.

- **FLASK_WORKERS**: número de procesos o hilos de trabajo que utilizará la aplicación.

6.3. Diseño Físico de Datos

El almacenamiento de datos en el sistema se realiza en dos niveles diferenciados. Por un lado, se utiliza una base de datos relacional para guardar la información estructurada de usuarios, ficheros, análisis, alertas y registros de actividad. Por otro, el sistema de archivos del propio sistema operativo, almacena las evidencias subidas y determinados recursos generados por el análisis.

Para la gestión de la base de datos se ha optado por emplear PostgreSQL¹ como motor de bases de datos relacional, gestionado mediante SQLAlchemy. Sin embargo, se permite utilizar SQLite² como alternativa ligera durante el desarrollo, o en entornos de prueba.

6.3.1. Sistema de archivos locales

Con la configuración por defecto, el sistema utiliza principalmente los siguientes directorios:

- **instance/uploads**: directorio donde se almacenan físicamente las evidencias subidas por los usuarios. Los ficheros se guardan con un nombre interno aleatorio y se asocian a su propietario a través de la base de datos.
- **app/static/avatars**: directorio donde se guardan las imágenes de perfil configuradas por los usuarios.
- **app/static/spectrograms**: directorio reservado para almacenar los espectrogramas generados a partir de análisis de ficheros de audio.
- Directorio externos de reglas: opcionalmente se usa un directorio externo para la gestión de las reglas YARA.

6.3.2. Base de datos

Para interactuar con la base de datos desde el aplicativo web se utiliza SQLAlchemy, integrado a través de Flask-SQLAlchemy. Esta capa de abstracción permite definir los modelos de datos como clases de Python y gestionar de forma sencilla las tablas, las relaciones y las consultas. La evolución del esquema se controla mediante migraciones gestionadas con Alembic³.

En la configuración habitual del sistema, la base de datos PostgreSQL definida por DATABASE_URL almacena las siguientes entidades principales:

¹PostgreSQL - <https://www.postgresql.org/>

²SQLite - <https://www.sqlite.org/>

³Alembic - <https://pypi.org/project/alembic/>

- User: almacena la información de las cuentas de usuario (identificador, nombre de usuario, contraseña en forma de hash, rol de administrador, URL del avatar y contador de notificaciones).
- File: representa cada fichero subido al sistema e incluye el usuario propietario, nombres original e interno, ruta de almacenamiento, tamaño, tipo, *mime type*, fecha de subida y hashes criptográficos principales (MD5, SHA1, SHA256).
- Analysis: reúne los resultados de cada análisis realizado sobre un fichero, como las salidas de ClamAV, YARA y VirusTotal, la presencia de macros o esteganografía, la puntuación de sandbox, el veredicto global y un resumen de los hallazgos, además de referencias al fichero y al usuario asociado.
- Alert: almacena las alertas generadas a partir de los análisis e indica severidad, título, descripción, estado de lectura y referencias al usuario, fichero y análisis correspondientes.
- Log: registra los eventos de auditoría del sistema (inicios y cierres de sesión, subidas de ficheros, finalización de análisis, cambios de configuración); junto con la marca temporal, el usuario implicado, la dirección IP, el *user-agent*, el tipo de acción y detalles adicionales.

Este diseño físico permite separar claramente la información estructurada y gestionada por la base de datos relacional del almacenamiento de ficheros binarios en el sistema de archivos, manteniendo así una organización coherente y eficiente.

6.4. Diseño de la Interfaz de Usuario

En esta sección se describirá el diseño de la interfaz de usuario del sistema, desde la fase de prototipado hasta la implementación final.

6.4.1. Prototipado

Para el diseño de la interfaz de usuario, se realizaron varios prototipos usando la herramienta de diseño Figma⁴.

Los prototipos sirvieron como una base para la implementación de la interfaz, permitiendo desarrollar de manera más rápida y eficiente.

En los siguientes apartados se muestran los resultados de la fase de implementación de la interfaz.

6.4.2. Elementos comunes

Primero se describirán los elementos comunes a todas las páginas del sistema.

⁴Figma - <https://www.figma.com/>

Encabezado superior (*header*)

Siguiendo el patrón de la mayoría de aplicaciones web modernas, todas las vistas del sistema comparten en la parte superior una barra de navegación (*top bar*) común. Esta barra proporciona acceso rápido a las secciones principales del sistema.

En la parte izquierda se sitúa la marca de la aplicación, compuesta por el logotipo de ForenAlyze y el nombre del proyecto. Este elemento actúa además como hipervínculo hacia la página de inicio (*home*), que funciona como punto de entrada principal a la aplicación.

En la parte derecha se agrupan los enlaces de navegación y los elementos contextuales.

De forma general, cuando el usuario no ha iniciado sesión, se muestran los siguientes enlaces:

- Home: enlace hacia la página de inicio pública, donde se presenta una descripción general de ForenAlyze y sus funcionalidades.
- Dashboard: enlace hacia el panel principal para usuarios autenticados. Redirige a la página de inicio de sesión si el usuario no ha iniciado sesión.
- About: enlace hacia la página con información detallada sobre el proyecto, su motivación y el contexto del TFM.
- Login: enlace hacia la página de inicio de sesión.

En cambio, cuando el usuario ha iniciado sesión, además de los enlaces anteriores; la sección de *Login* se sustituye por un icono de campana que indica el número de notificaciones pendientes, y un menú desplegable con el nombre y la foto de perfil del usuario. Este menú ofrece opciones diferentes según el rol del usuario:

- Para usuarios estándar: opciones para editar el perfil y para cerrar sesión.
- Para usuarios administradores: opciones para acceder al apartado de gestión de usuarios y las dos opciones incluidas para usuarios estándar.



Figura 6.8: Barra de navegación superior - usuario no autenticado



Figura 6.9: Barra de navegación superior - usuario autenticado (administrador)



Figura 6.10: Barra de navegación superior - usuario autenticado (estándar)

Barra inferior (*footer*)

Continuando con el anterior patrón, todas las páginas del sistema comparten en la parte inferior una barra común. Esta barra aparece fija al final del contenido y actúa como cierre visual de la interfaz.

El *footer* incluye el nombre de la aplicación, el año de creación y una breve referencia al contexto académico del proyecto (Trabajo Fin de Máster de la Universidad de Cádiz).

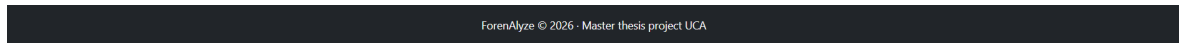


Figura 6.11: Barra inferior

6.4.3. Página principal

La página principal actúa como punto de entrada al sistema y está orientada a presentar la herramienta al usuario antes de acceder a las funcionalidades avanzadas.

La estructura de esta vista se organiza en tres bloques principales. En primer bloque muestra una sección de portada en dos columnas: a la izquierda aparece el nombre de la aplicación, una breve descripción y un botón de acción que dirige al usuario autenticado hacia su panel de control (*dashboard*); a la derecha se incluye una imagen ilustrativa relacionada con la aplicación. Ambos elementos incorporan animaciones suaves, implementadas mediante la librería AOS.

El segundo bloque presenta una sección titulada "*What can you do with ForenAlyze?*", formada por varias tarjetas informativas que resumen las capacidades del sistema.

El tercer bloque incluye un apartado de *FAQ* (preguntas frecuentes) implementado mediante un componente de acordeón. Este bloque responde a las dudas más comunes que pueden surgir a los usuarios, como la seguridad de los datos, el funcionamiento de los análisis o la compatibilidad con distintos tipos de ficheros.

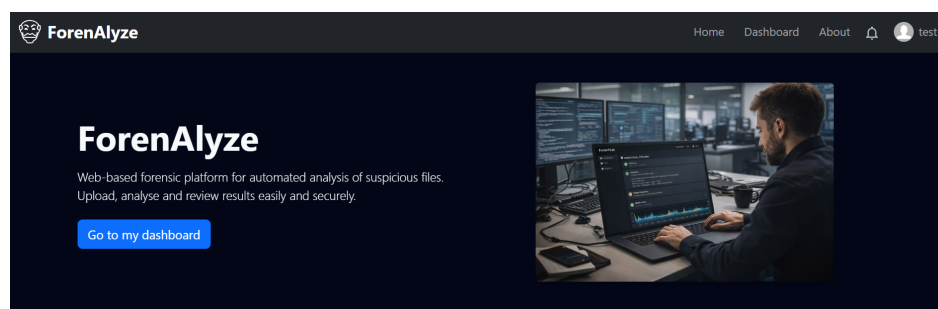


Figura 6.12: Página principal - Portada

6.4.5. Página de inicio de sesión

Para acceder al sistema es necesario autenticarse mediante la página de inicio de sesión, accesible desde el enlace *Login* de la barra de navegación superior.

La vista presenta un formulario sencillo con dos campos: *username* (nombre de usuario) y *password* (contraseña); además de un botón para enviar las credenciales. Si el usuario introduce un nombre de usuario o una contraseña no válidos, el sistema muestra un mensaje de error informando del fallo en la autenticación.

En el diseño actual no existe un formulario de registro público, las cuentas de usuario son creadas y gestionadas por los administradores desde el panel de administración.

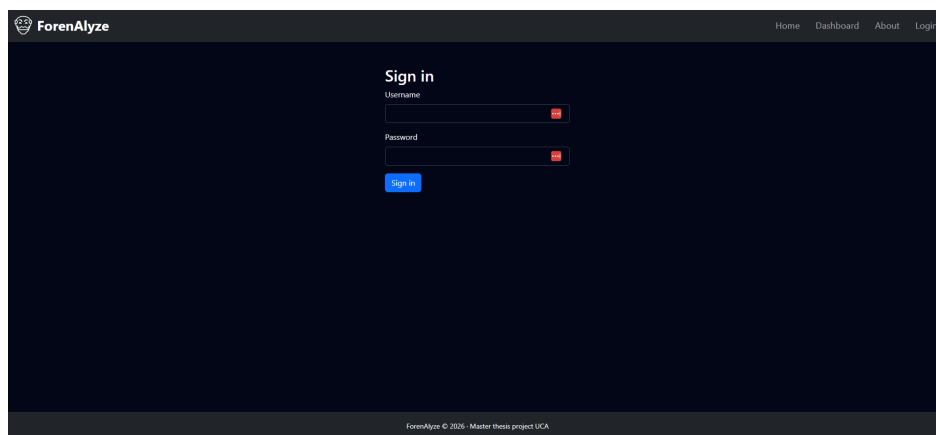


Figura 6.16: Página de inicio de sesión

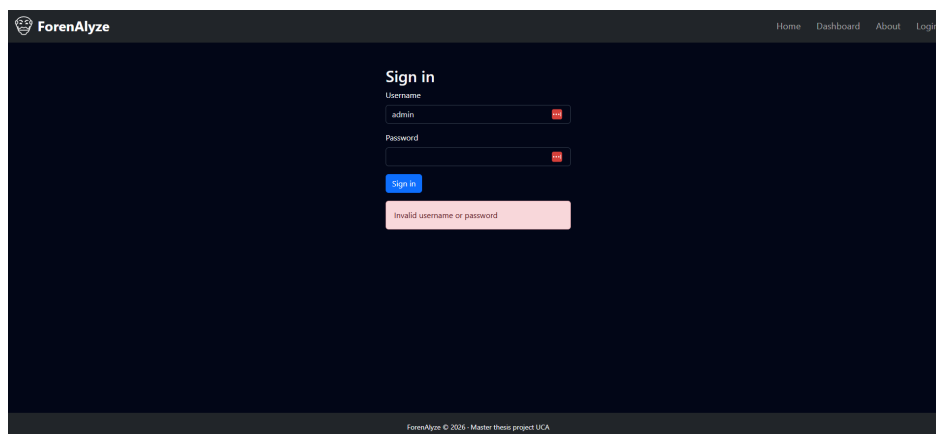


Figura 6.17: Página de inicio de sesión - Error

6.4.6. Página de panel de control (*dashboard*)

La página de *dashboard* constituye la vista principal para los usuarios autenticados y resume el estado de la actividad reciente en la plataforma. Es accesible desde el enlace

Dashboard del menú superior y es el punto de partida para acceder a las funcionalidades del aplicativo.

Además del encabezado general, esta vista incorpora un encabezado secundario (*subheader*), específico del módulo de análisis que presenta un menú de pestañas con acceso rápido a las secciones más utilizadas: *Dashboard*, *Files & analyses*, *Logs*, *Storage* y *YARA rules*, junto con un enlace directo a *Upload file*. Este menú facilita la navegación entre las distintas funcionalidades sin abandonar el contexto del área forense.

La interfaz se organiza en varios bloques: en la parte superior se muestran indicadores clave (*KPIs*); como el número total de ficheros subidos, los análisis realizados, los pendientes y el volumen de ficheros maliciosos detectados en las últimas horas. A continuación, se presentan gráficos con la evolución temporal de los veredictos, la distribución por tipos de fichero y el origen de las detecciones (ClamAV, YARA, *sandbox*, macros o esteganografía). Los gráficos han sido creados usando la biblioteca de gráficos *Chart.js*⁵. Finalmente, se incluyen tablas con los ficheros analizados más recientes y con las alertas más relevantes, desde las que es posible acceder al informe detallado de cada análisis.

Es importante destacar que el ámbito de los datos visualizados depende del rol del usuario. Los usuarios con perfil de administrador pueden consultar métricas globales de toda la aplicación, mientras que los usuarios estándar sólo ven estadísticas y listados relativos a sus propios ficheros y análisis, respetando así el principio de mínima revelación de información.

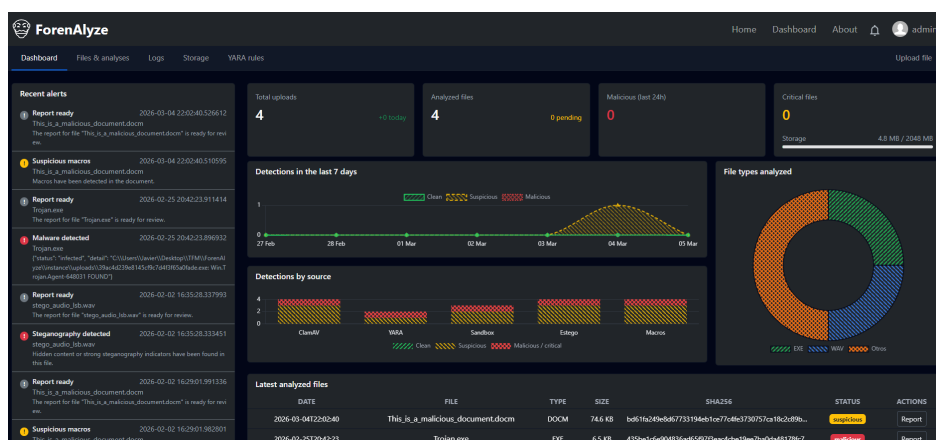


Figura 6.18: Página de panel de control (*dashboard*)

6.4.7. Página de subida de ficheros

La página de subida es la vista donde los usuarios pueden cargar archivos para su análisis. Esta se centra en un área de tipo *drag & drop*, donde puede arrastrar directamente el fichero a analizar. Alternativamente, puede pulsar el botón “*Choose file*” para abrir el selector estándar del sistema operativo. En ambos casos, el nombre del archivo seleccionado se muestra en la propia interfaz como confirmación visual.

⁵Chart.js - <https://www.chartjs.org/>

Bajo la zona de subida se indican de forma explícita las restricciones más relevantes: tipos de fichero permitidos (ejecutables, documentos ofimáticos, PDF, imágenes y audio WAV, entre otros), y tamaño máximo de 100MB. El formulario advierte también que no deben subirse datos personales o confidenciales.

Antes de enviar el formulario se aplican validaciones básicas en el navegador (extensión y tamaño del fichero), y de nuevo en el servidor. Si el usuario intenta subir un archivo vacío, de tipo no admitido o que supera el límite de tamaño, la interfaz muestra mensajes de error claros mediante cuadros de diálogo.

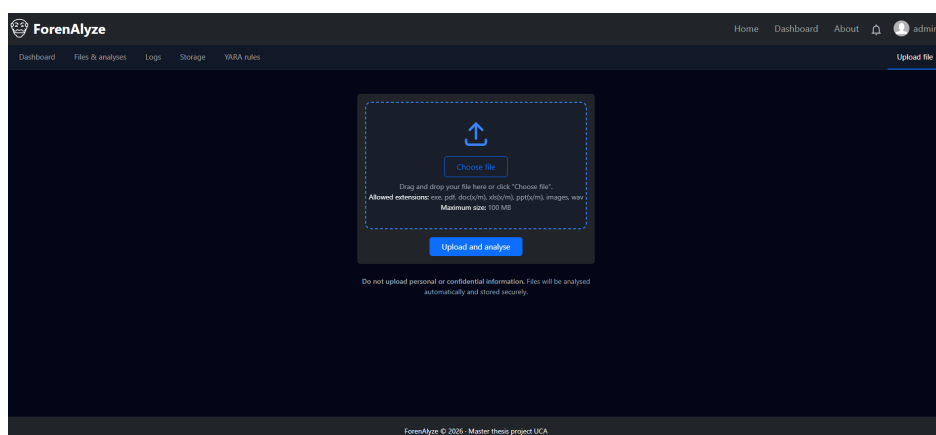


Figura 6.19: Página de subida de ficheros

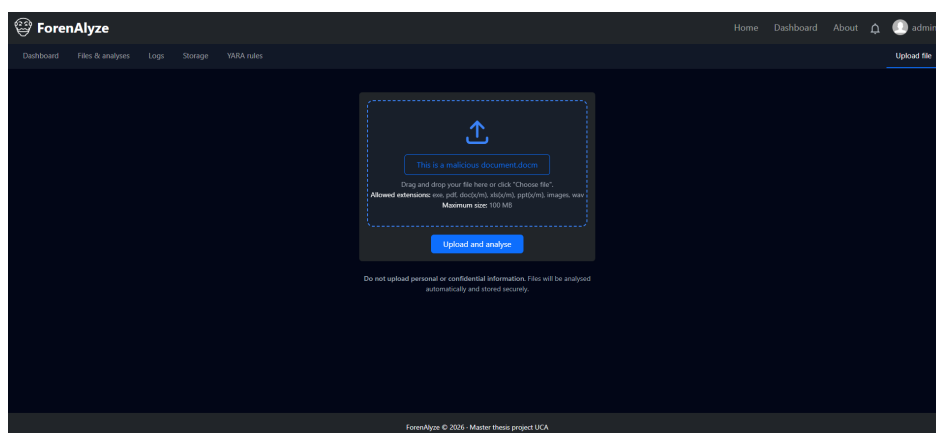


Figura 6.20: Página de subida de ficheros - fichero seleccionado

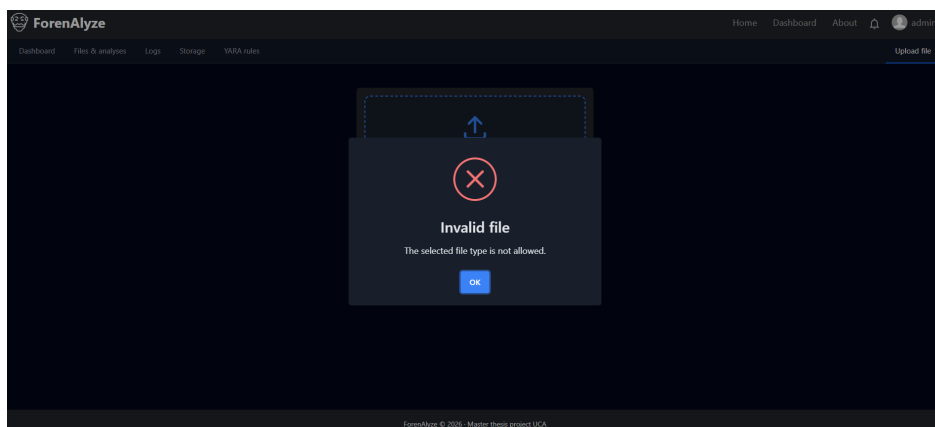


Figura 6.21: Página de subida de ficheros - error: tipo de fichero no permitido

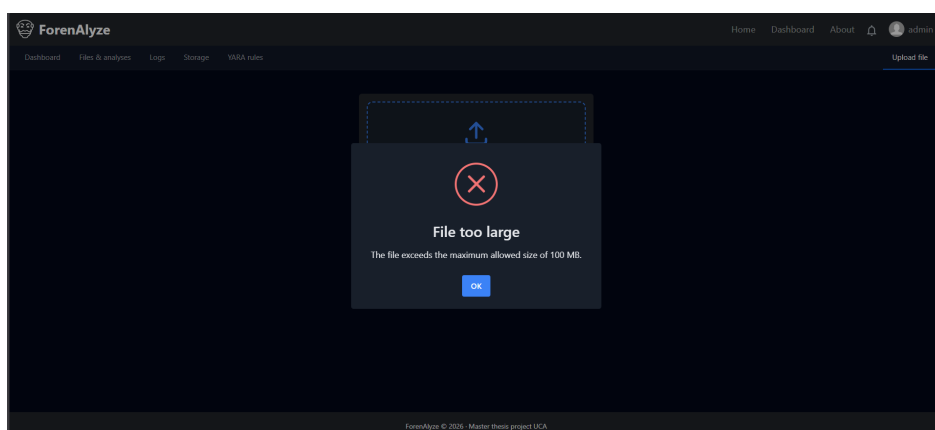


Figura 6.22: Página de subida de ficheros - error: supera el tamaño máximo permitido

6.4.8. Página de ficheros y análisis

La página de ficheros y análisis ofrece una vista de los documentos subidos y de los resultados de los análisis realizados por el *pipeline*. Es accesible desde el subheader del módulo de análisis a través de la opción *Files & analyses* y se estructura en dos pestañas principales: una centrada en los ficheros almacenados y otra en los análisis completados.

En la pestaña *Files* se muestra una tabla con todos los ficheros registrados e incluye el nombre original, el tipo, el tamaño, el veredicto actual y la fecha de subida. El veredicto se representa mediante insignias de color (verde para ficheros limpios, amarillo para sospechosos, rojo para maliciosos o críticos). Para cada entrada se ofrece un botón de información que abre un cuadro modal con los metadatos detallados del fichero, como los nombres interno y externo; el tipo y MIME, los hashes (MD5, SHA1, SHA256), la ruta de almacenamiento y el usuario que realizó la subida. En caso de que todavía no existan ficheros, la página muestra un mensaje informativo y un acceso directo a la pantalla de subida.

La pestaña *Analyses* presenta una tabla orientada al seguimiento de los resultados

forenses. Cada fila corresponde a un análisis concreto e incluye la fecha de ejecución, el fichero asociado, el tipo y tamaño, el hash SHA256, el veredicto global y el usuario que lo ha lanzado. Además, se muestran pequeñas insignias que indican qué motores han intervenido (ClamAV, YARA, detección de macros, esteganografía o análisis de audio), facilitando identificar rápidamente el origen de las detecciones. Desde esta vista, se puede acceder directamente al informe detallado de cada análisis mediante un enlace específico.



Figura 6.23: Página de ficheros y análisis - pestaña Files

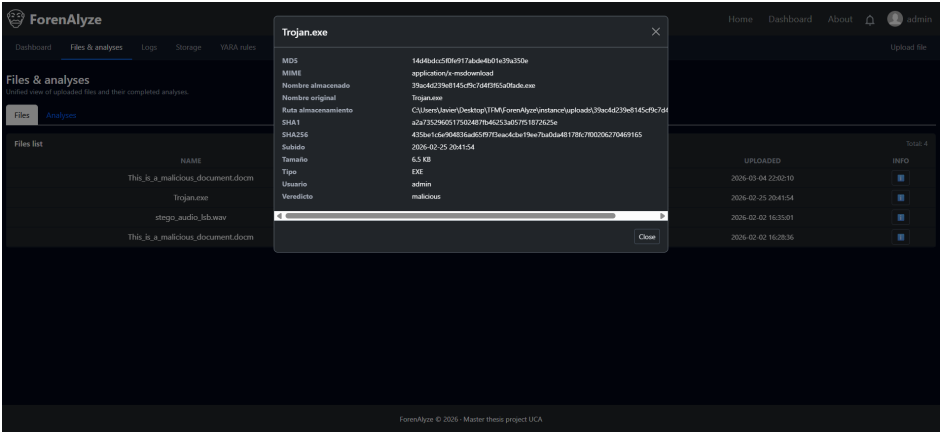


Figura 6.24: Página de ficheros y análisis - pestaña Files (modal de metadatos)

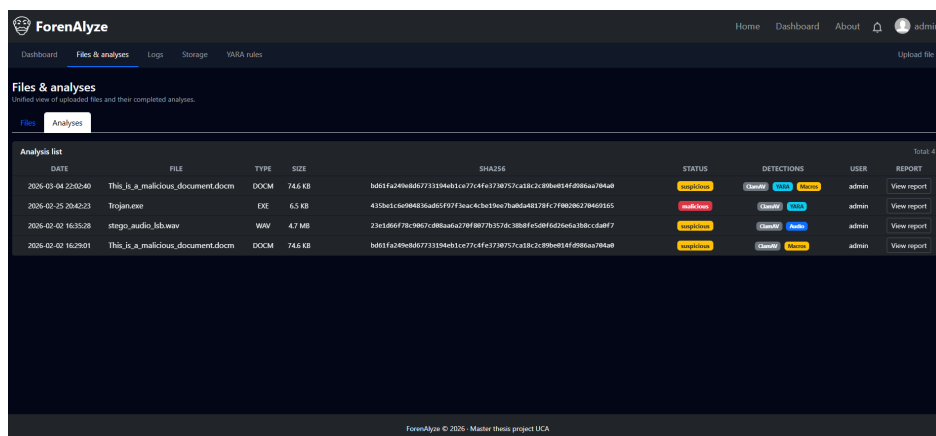


Figura 6.25: Página de ficheros y análisis - pestaña Analyses

6.4.9. Página de informe de análisis

La página de informe de análisis se accede desde la tabla de *Files & analyses* mediante el botón *View report* asociado al informe de un fichero específico. Desde el *subheader*, es posible descargar el informe en formato JSON o PDF.

El informe se organiza en cuatro apartados principales, cada uno con su propia estructura y contenido específico.

Apartado 1. Información general y veredicto

En este apartado se presenta la información básica del fichero analizado: nombre original, tipo/MIME, tamaño, usuario que lo ha subido, fechas de subida y análisis, ruta de almacenamiento y huellas MD5/SHA1/SHA256. Se muestra también el veredicto global y un breve resumen del análisis realizado.

Apartado 2. Motores y reglas

Aquí se agrupan los resultados de ClamAV y de las reglas YARA aplicadas. Se indica el estado del motor (limpio, infectado, error, no disponible), y para las reglas YARA, el listado de reglas que han coincidido con sus espacios de nombres, etiquetas y metadatos técnicos.

Apartado 3. Inteligencia externa

Este bloque recoge la información procedente de servicios de inteligencia externos, fundamentalmente VirusTotal, y una *sandbox* de terceros como Hybrid Analysis. Para VirusTotal se muestran las estadísticas de los motores (número de detecciones, distribución por tipo de veredicto) y un enlace directo al reporte público asociado al *hash* del fichero. En el caso de Hybrid Analysis el informe incluye su veredicto, la posible familia de *malware*, las etiquetas asignadas y un breve resumen.

Apartado 4. Contenido y módulos avanzados

En este apartado se incluyen módulos adicionales que sólo aparecen cuando el tipo de fichero lo permite o cuando se dispone de información extra:

- Análisis detallado de macros VBA en documentos Office con macros (incluyendo indicadores sospechosos y módulos descargables).
- Extracción de texto y metadatos mediante Apache Tika para documentos PDF y Office.
- Metadatos y espectrograma de audio para ficheros de sonido.
- Bloque específico de esteganografía para imágenes o audio donde se muestran posibles mensajes ocultos y *payloads* base64 con sus previsualizaciones.

Cuando alguno de los elementos no está disponible, la interfaz muestra mensajes explícitos indicando que no se dispone de datos para ese caso concreto.

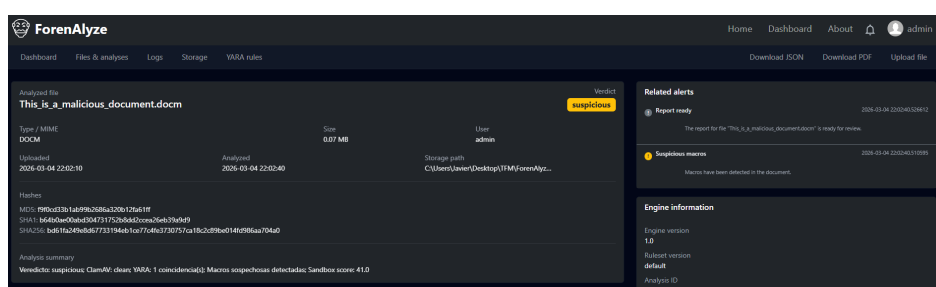


Figura 6.26: Página de informe de análisis - Información general y veredicto

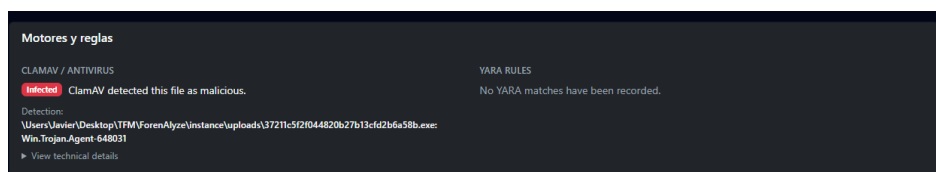


Figura 6.27: Página de informe de análisis - Motores y reglas

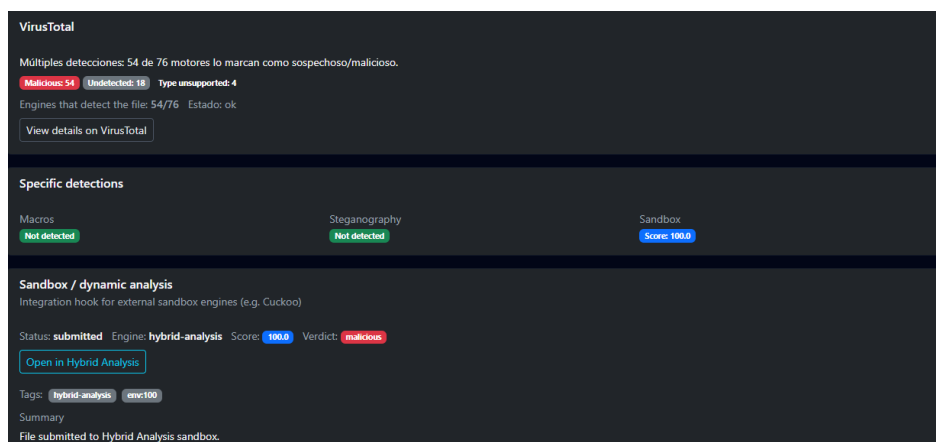
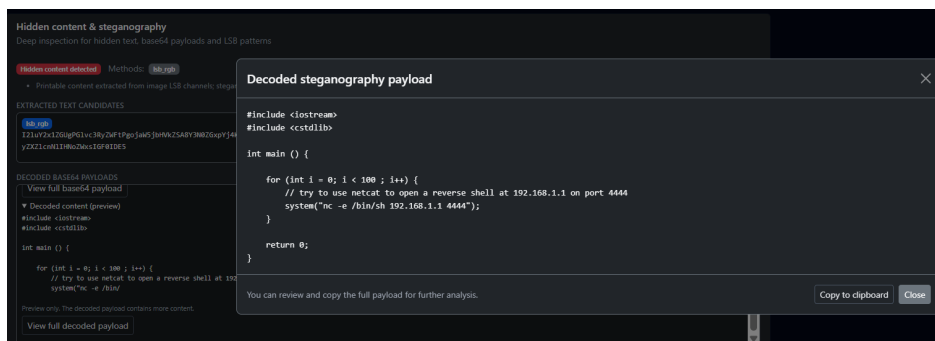
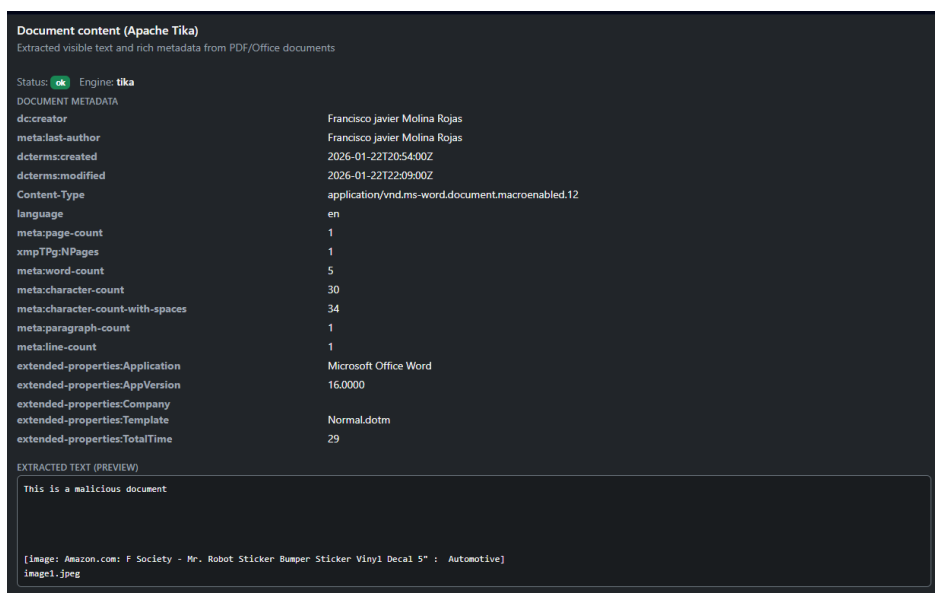
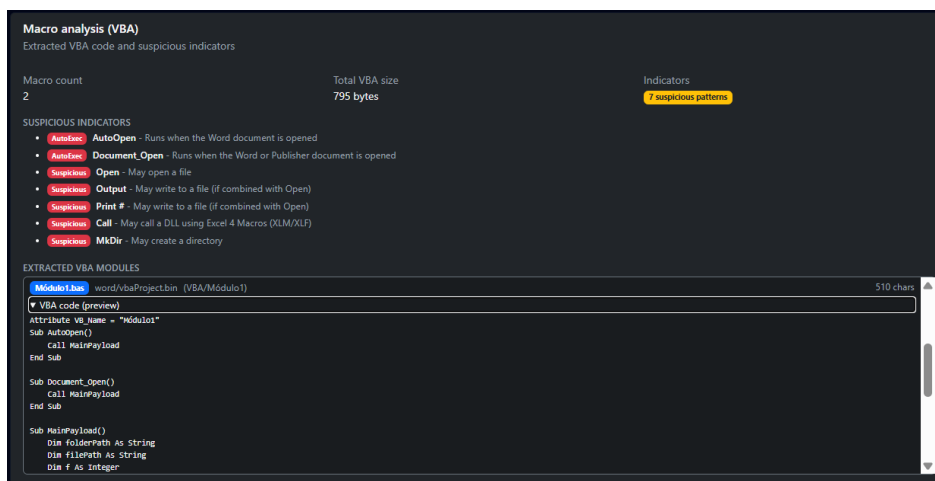


Figura 6.28: Página de informe de análisis - Inteligencia externa (VirusTotal y Hybrid Analysis)



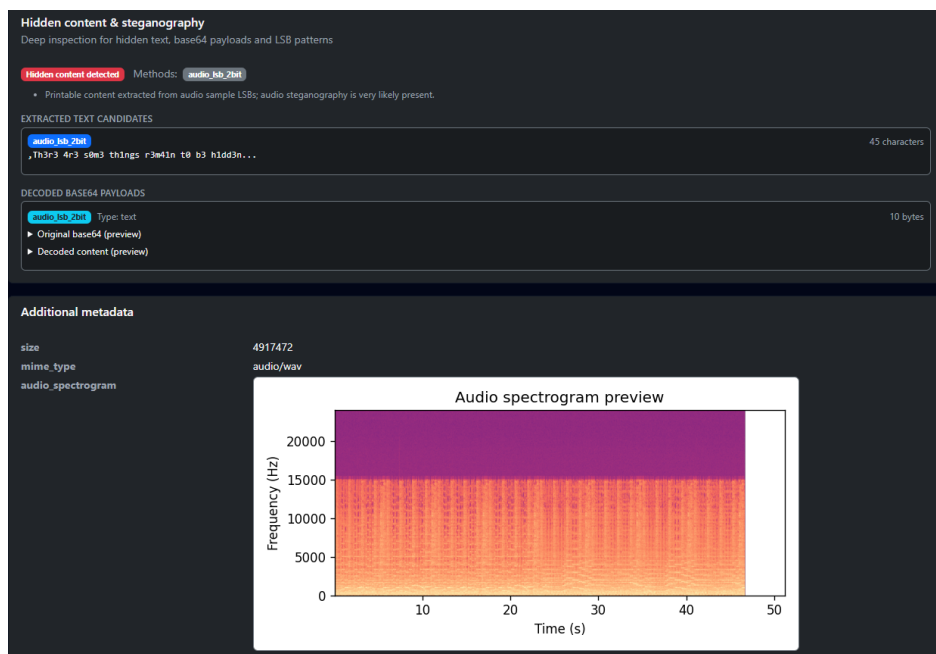


Figura 6.32: Página de informe de análisis - Módulo de esteganografía en audio

6.4.10. Página de *Logs*

La página de *Logs* ofrece un registro centralizado de la actividad relevante en la plataforma (inicios de sesión, subidas de ficheros, ejecuciones de análisis y cambios de configuración). La vista muestra una tabla paginada con la fecha y hora del evento, el usuario implicado, la acción realizada, el recurso afectado, el estado, la dirección IP y un mensaje descriptivo; además de un botón que permite abrir un modal con los detalles técnicos en formato JSON cuando están disponibles.

La información mostrada depende del rol del usuario. Los administradores pueden revisar el historial completo de eventos de todos los usuarios; en cambio, los usuarios estándar sólo ven los *logs* asociados a su propia actividad.

ForenAnalyze

DashboardFiles & analysesLogsStorageYARA rules

HomeDashboardAboutadmin

Upload file

Activity logs

Centralised audit trail for key actions performed in ForenAnalyze

Logs

TIMESTAMP	USER	ACTION	RESOURCE	STATUS	IP	MESSAGE	DETAILS
2026-03-05 12:18:23	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'stego_image_2.png'.	
2026-03-05 12:18:11	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'stego_image_1.png'.	
2026-03-05 12:17:57	admin	upload	dashboard:upload	success	127.0.0.1	File 'stego_image_2.png' uploaded and queued for analysis.	
2026-03-05 12:17:49	admin	upload	dashboard:upload	success	127.0.0.1	File 'stego_image_1.png' uploaded and queued for analysis.	
2026-03-05 12:13:07	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'Trojanuser'.	
2026-03-05 12:11:42	admin	upload	dashboard:upload	success	127.0.0.1	File 'Trojanuser' uploaded and queued for analysis.	
2026-03-05 12:10:53	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'Trojanuser'.	
2026-03-05 12:10:21	admin	upload	dashboard:upload	success	127.0.0.1	File 'Trojanuser' uploaded and queued for analysis.	
2026-03-05 12:04:40	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'Trojanuser'.	
2026-03-05 12:04:01	admin	upload	dashboard:upload	success	127.0.0.1	File 'Trojanuser' uploaded and queued for analysis.	
2026-03-04 22:02:40	admin	analysis_completed	dashboard:analysis_background	success	-	Analysis completed for file 'This_is_a_malicious_document.docm'.	
2026-03-04 22:02:10	admin	upload	dashboard:upload	success	127.0.0.1	File 'This_is_a_malicious_document.docm' uploaded and queued for analysis.	
2026-03-04 22:01:17	admin	login	auth:login	success	127.0.0.1	User logged in successfully.	
2026-03-04 21:58:22	N/A	login_failed	auth:login	failed	127.0.0.1	Failed login attempt.	
2026-03-04 21:58:03	N/A	login_failed	auth:login	failed	127.0.0.1	Failed login attempt.	

Page 1 of 2

Figura 6.33: Página de *Logs*

6.4.11. Página de gestión del almacenamiento

La página de *Storage* resume el uso del espacio en disco asociado al usuario autenticado. Asimismo, muestra una tarjeta con el consumo actual frente a la cuota configurada (barra de progreso y porcentaje), un mensaje de estado (normal, aviso o lleno) y una serie de recomendaciones para liberar espacio.

Bajo estas métricas se presenta una tabla con los ficheros del usuario, indicando: nombre, tamaño, veredicto y fecha de subida, junto con una acción de borrado que permite eliminar el fichero y sus informes asociados después de una confirmación por parte del usuario.

A diferencia de otras vistas, esta pantalla está centrada exclusivamente en el almacenamiento personal.

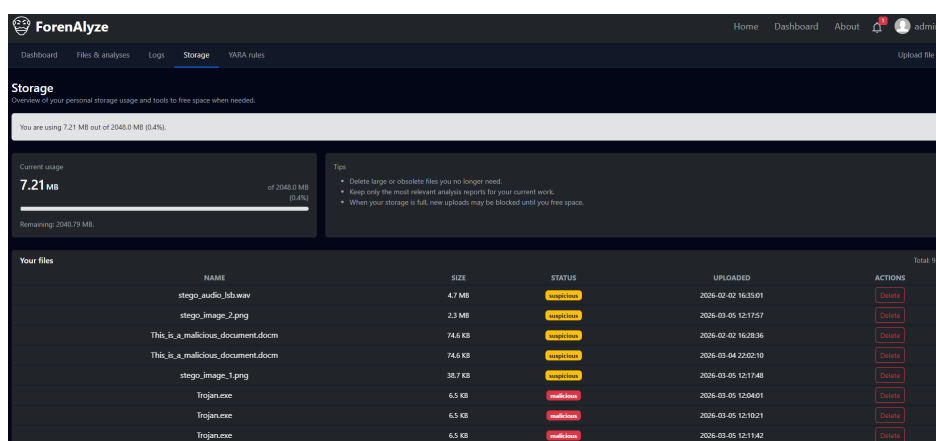


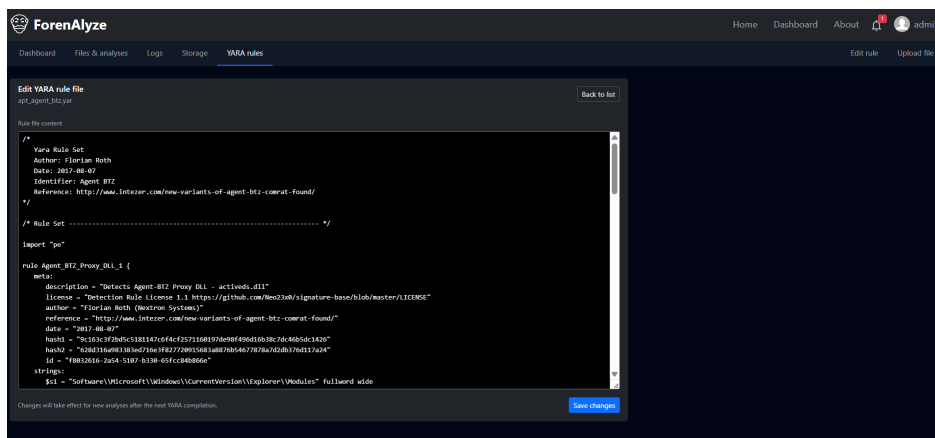
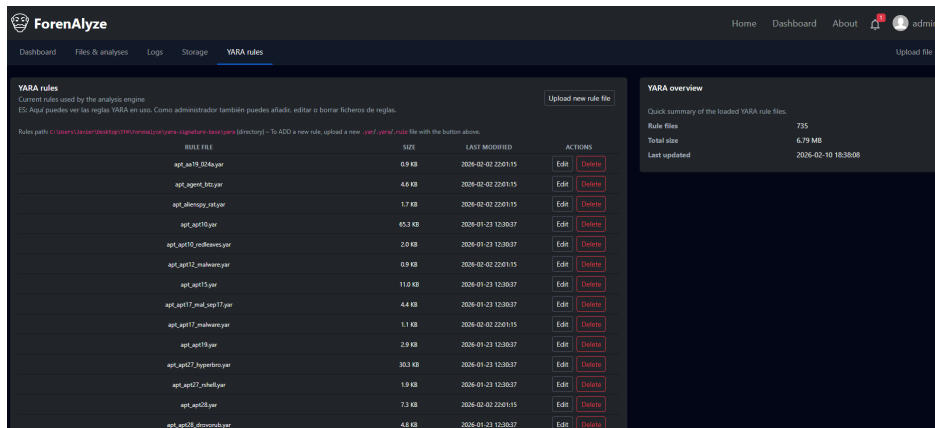
Figura 6.34: Página de gestión del almacenamiento

6.4.12. Página de gestión de reglas YARA

La página de gestión de reglas YARA muestra en una tabla todos los ficheros de reglas cargados desde la ruta configurada en el sistema, indicando: nombre, tamaño y fecha de última modificación. Además, se incluye un pequeño resumen del número total de ficheros y del espacio que ocupan.

Los usuarios sin privilegios de administración pueden consultar este listado en modo solo lectura; mientras los administradores disponen además de acciones para subir nuevos ficheros de reglas, editar los existentes o eliminarlos.

Cuando el administrador edita una regla, el editor de reglas YARA se presenta como una vista de edición de texto. Antes de guardar los cambios, o aceptar un nuevo fichero subido; el sistema valida la sintaxis mediante el motor *yara-python* y si se detectan errores, el guardado se bloquea mostrando mensajes explicativos para que el usuario corrija la regla.



6.4.13. Página de gestión de usuarios

La página de gestión de usuarios está disponible para administradores en el *drop-down* de su perfil permitiendo crear nuevas cuentas, consultar las existentes y realizar tareas de mantenimiento básicas.

En la parte izquierda se muestra un formulario para dar de alta usuarios; donde se especifican nombre de usuario, contraseña y si el nuevo perfil tendrá privilegios de administración. En la parte derecha aparece una tabla con todos los usuarios registrados, indicando su rol y el espacio de almacenamiento que están consumiendo, junto con acciones para cambiar la contraseña o eliminar la cuenta en caso de no ser administrativa.

Los administradores pueden reiniciar la contraseña de otros usuarios e incluso borrar cuentas completas; operación que implica eliminar también sus ficheros, análisis, alertas y *logs* asociados, liberando el espacio ocupado en el sistema.

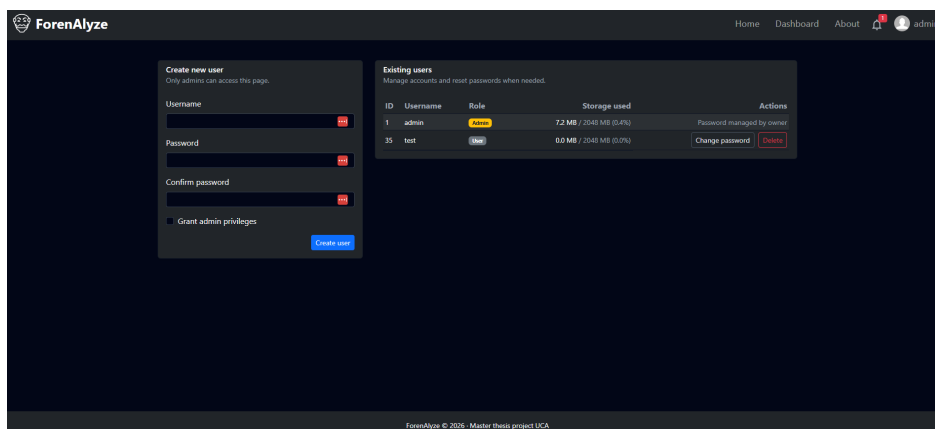


Figura 6.37: Página de gestión de usuarios

6.4.14. Página de edición de perfil

La página de edición de perfil está disponible para todos los usuarios en el *dropdown* de su perfil y permite gestionar su propia cuenta sin depender de un administrador. Desde esta página se puede actualizar el nombre visible dentro de la plataforma, cambiar la fotografía de perfil y modificar su contraseña.

La interfaz se organiza en tres apartados principales. En la parte superior se muestra el nombre de usuario actual permitiendo editarlo. A la izquierda se muestra la fotografía de perfil actual y un botón para cambiarla. Por último, en la parte derecha se incluye un bloque para cambiar la contraseña.

El cambio de contraseña se realiza siempre desde esta página y exige introducir la contraseña actual junto con la nueva y su confirmación.

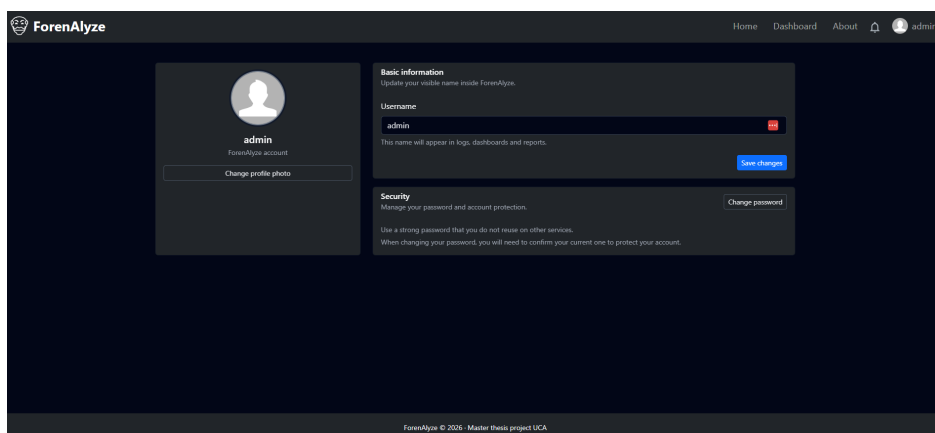


Figura 6.38: Página de edición de perfil

7. Construcción del Sistema

En este capítulo se describe el proceso de construcción del sistema, incluyendo: el entorno de construcción, el código fuente y detalles de implementación.

7.1. Entorno de Construcción

En el desarrollo de la plataforma se ha utilizado como lenguaje principal Python; aprovechando su ecosistema de librerías para análisis forense, seguridad y desarrollo web. El *framework* escogido ha sido Flask, debido a que el proyecto requería una aplicación web ligera y fácil de desplegar.

Además, se ha trabajado sobre un entorno virtual de Python; de forma que las dependencias del proyecto quedan aisladas del sistema operativo. La gestión de paquetes se realiza mediante `pip` y un fichero `requirements.txt`, lo que facilita la instalación de las dependencias necesarias.

Como entorno de desarrollo integrado (IDE) se ha utilizado principalmente Visual Studio Code, junto con extensiones específicas para Python y soporte de Docker. El proyecto se encuentra disponible en un repositorio en GitHub¹, lo que permite disponer de control de versiones y una copia de seguridad remota del código fuente.

Para crear los servicios auxiliares necesarios (base de datos PostgreSQL, servidor Apache Tika para extracción de texto y metadatos, motor ClamAV), se utiliza Docker y `docker-compose`. A través de un único fichero `docker-compose.yml` se definen y levantan los contenedores necesarios, lo que reduce el esfuerzo de instalación manual de dependencias externas.

Finalmente, para la creación de los diagramas de arquitectura, modelo de datos y flujos de interacción se han utilizado herramientas de modelado en línea como draw.io² y Visual Paradigm Online³.

7.2. Código Fuente

El código fuente de la aplicación se organiza en varios ficheros y directorios principales en la raíz del proyecto:

- `run.py`: punto de entrada para ejecutar el sistema. Crea la aplicación Flask a partir de la factoría `create_app` y lanza el servidor, ajustando el número de hilos o procesos según la plataforma.

¹Repositorio de GitHub de ForenAlyze - <https://github.com/Javivi-MR/ForenAlyze>

²draw.io - <https://app.diagrams.net/>

³Visual Paradigm Online - <https://online.visual-paradigm.com/>

- **create_admin.py**: script auxiliar que permite crear desde consola un usuario administrador inicial.
- **Dockerfile** y **docker-compose.yml**: ficheros de definición de la imagen de la aplicación y de los servicios externos (base de datos, Tika, ClamAV).
- **requirements.txt**: listado de dependencias Python necesarias para ejecutar la aplicación y el *pipeline* de análisis.

El núcleo de la aplicación se encuentra en el directorio **app/**, que contiene los módulos principales:

- **app/__init__.py**: define la función **create_app**, que es la encargada de crear la instancia de Flask, cargar la configuración y inicializar las extensiones (base de datos, migraciones, autenticación).
- **app/config.py**: contiene las clases de configuración y lee variables de entorno para parámetros como la URL de la base de datos, la clave secreta, la cuota de almacenamiento...
- **app/extensions.py**: inicializa las extensiones de Flask utilizadas en el proyecto (Flask-SQLAlchemy, Flask-Migrate, Flask-Login...).
- **app/models.py**: define los modelos de la base de datos (usuarios, ficheros, análisis, alertas y *logs*), así como sus relaciones y campos específicos.
- **app/auth/**: contiene el módulo de autenticación, con las rutas de inicio y cierre de sesión y la integración con Flask-Login.
- **app/routes/**: agrupa las vistas principales de la interfaz.
- **app/analysis/**: implementa el *pipeline* de análisis forense. Incluye funciones para calcular hashes, ejecutar ClamAV, aplicar reglas YARA, consultar VirusTotal, integrar una *sandbox* externa, extraer metadatos con Apache Tika, analizar macros de Office y realizar heurísticas de esteganografía y análisis de audio.
- **app/services/**: contiene servicios como la generación de alertas o el registro de eventos en la tabla de *logs*.

Además, el proyecto incluye varios directorios de recursos y datos:

- **instance/uploads**: almacena los ficheros subidos por los usuarios.
- **app/static/**: contiene recursos estáticos como archivos JavaScript (**app/static/js/**), imágenes de perfil (**app/static/avatars/**) y espectrogramas de audio generados (**app/static/spectrograms/**).
- **app/templates/**: agrupa las plantillas Jinja2 utilizadas por la interfaz web. Incluye las plantillas de inicio (**home.html**), información del proyecto (**about.html**), autenticación (**login.html**), panel de control (**dashboard.html**), subida de ficheros (**upload.html**), listado de ficheros y análisis (**files.html**), *logs* (**logs.html**), gestión de almacenamiento (**storage.html**), informe de análisis (**analysis_report.html**), gestión de usuarios (**users.html**) y gestión de reglas YARA (**yara_rules.html** y **yara_rule_edit.html**). La plantilla **base.html** define la estructura común (cabecera, menús y pie) para el resto.

- **migrations/**: directorio gestionado por Flask-Migrate/Alembic que almacena las migraciones.
- **tests/**: contiene las pruebas automatizadas que usan *pytest*. Estas validan el flujo básico de autenticación, subida y análisis de ficheros y la creación de alertas y *logs*.
- **yara-signature-base/**: copia del repositorio público de reglas YARA utilizado como base de firmas.

7.2.1. Detalles de la hoja de estilos

Para la construcción de la interfaz se ha usado el *framework* Bootstrap⁴, lo que ha agilizado la implementación. Sobre esta base se ha utilizado un tema oscuro, más acorde con las herramientas de análisis y seguridad.

Además, se ha usado la librería AOS⁵ (*Animate On Scroll*) para añadir animaciones suaves y SweetAlert2⁶ para mostrar diálogos de confirmación y mensajes de error de forma más atractiva.

7.2.2. Manejo de datos y creación de gráficos

El manejo de los ficheros a analizar se realiza a través de los modelos de datos *File* y *Analysis*, que almacenan tanto los metadatos (nombre, tipo, tamaño, hashes, rutas internas), como los resultados generados del *pipeline* (veredictos, resultados de motores, resúmenes). Los ficheros físicos se guardan en el directorio **instance/uploads**, mientras que los análisis se almacenan en la base de datos relacional.

Para la representación gráfica en el *dashboard* se utiliza la librería JavaScript Chart.js⁷, que consume datos por la API interna de la propia aplicación. Con ella se generan gráficos de líneas y barras que muestran la evolución de detecciones en los últimos días, la distribución de tipos de fichero analizados y el origen de las detecciones.

En el caso de ficheros de audio, los espectrogramas se generan mediante librería Matplotlib⁸ de Python.

7.3. Scripts de Base de Datos

La gestión del esquema de la base de datos de ForenAlyze utiliza Flask-Migrate y Alembic. A partir de los modelos definidos en **app/models.py**, se generan y aplican migraciones que permiten evolucionar la estructura de tablas sin necesidad de recrear

⁴Bootstrap - <https://getbootstrap.com/>

⁵AOS - <https://michalsnik.github.io/aos/>

⁶SweetAlert2 - <https://sweetalert2.github.io/>

⁷Chart.js - <https://www.chartjs.org/>

⁸Matplotlib - <https://matplotlib.org/>

la base de datos desde cero. Las migraciones se almacenan en el directorio `migrations/` y pueden ejecutarse mediante comandos estándar de Flask.

7.3.1. Inicialización y creación de usuarios

En vez de crear la base de datos manualmente, la aplicación está preparada para inicializarla cuando se despliega por primera vez. Adicionalmente, el script `create_admin.py` facilita la creación del primer usuario administrador desde línea de comandos.

7.3.2. Limpieza y consistencia de datos

Las operaciones administrativas como la eliminación de usuarios, o la gestión del almacenamiento incluyen lógica específica para mantener la consistencia de la base de datos. Por ejemplo, al borrar un usuario desde la página de administración se eliminan también sus ficheros asociados, los análisis correspondientes, las alertas generadas y los registros de *logs* vinculados a su actividad.

8. Pruebas del Sistema

En este capítulo se describen las pruebas realizadas al sistema, incluyendo: la estrategia de pruebas, los casos de prueba y los resultados obtenidos.

8.1. Estrategia

La estrategia de pruebas tiene tres objetivos principales: comprobar la corrección funcional de los flujos críticos, validar los mecanismos de seguridad y garantizar la no regresión ante cambios en el código.

Para ello se ha optado por la automatización de pruebas a nivel de servidor utilizando el *framework* `pytest`¹ y el cliente de pruebas de Flask.

Las pruebas se organizan en una jerarquía que combina:

- Pruebas unitarias y funcionales sobre vistas y servicios, que validan el comportamiento de rutas individuales (por ejemplo, acceso a páginas públicas, inicio de sesión, cierre de sesión o visualización de informes).
- Pruebas de integración que comprueban flujos completos entre varios componentes (por ejemplo, subida de un fichero, creación de entradas en base de datos, generación de análisis y alertas o registro de *logs*).
- Pruebas de sistema y aceptación realizadas en un entorno lo más parecido al de producción (contenedores Docker con PostgreSQL, ClamAV, Apache Tika y motor YARA).

Esta pirámide de pruebas se complementa con integración continua mediante GitHub Actions. Se han definido flujos de trabajo que se ejecutan cuando se realizan cambios en el repositorio. Estos instalan las dependencias del proyecto, ejecutan la batería de pruebas con `pytest`, y lanzan análisis de calidad y seguridad del código usando la herramienta SonarQube².

8.2. Pruebas Unitarias y Funcionales

Las pruebas unitarias y funcionales se apoyan en una aplicación Flask de prueba, una base de datos aislada en memoria y un cliente HTTP simulado. Sobre este entorno controlado se ejercitan las rutas principales sin necesidad de levantar el servidor real ni un navegador web, reduciendo el tiempo de ejecución y la complejidad de las pruebas.

¹`pytest` - <https://pytest.org/>

²SonarQube - <https://www.sonarqube.org/>

En primer lugar, se incluyen pruebas básicas de accesibilidad en las páginas públicas. Se verifica que la página principal y la sección de información están disponibles y devuelven códigos de estado correctos, lo que sirve como prueba rápida para comprobar que la aplicación arranca y las plantillas esenciales se muestran sin errores.

A continuación, se definen pruebas funcionales sobre el flujo de autenticación; para ello, se crean usuarios de prueba en la base de datos con contraseñas cifradas y se comprueba que:

- Un usuario inexistente o con credenciales incorrectas no puede acceder al sistema.
- Un usuario válido puede iniciar sesión y es redirigido correctamente al panel de control (*dashboard*).
- Al cerrar sesión, la sesión actual deja de ser válida y se redirige al usuario a la pantalla de *Login*.

Estas pruebas garantizan tanto la corrección de la lógica de autenticación como la navegación esperada desde el punto de vista del usuario.

Se contemplan también pruebas funcionales relacionadas con la gestión de usuarios y roles. Por ejemplo, se verifica que un administrador autenticado puede crear nuevos usuarios mediante las vistas de administración y que un usuario sin privilegios no puede acceder a dichas vistas y es redirigido al panel principal. De esta forma se comprueba que la separación de privilegios entre usuarios normales y administradores se implementa de manera consistente.

8.3. Pruebas de Integración

Las pruebas de integración se centran en flujos que atraviesan varios componentes de la aplicación: modelos de base de datos, lógica de análisis, generación de alertas, registro de *logs* y vistas de administración.

Desde el inicio del desarrollo se han diseñado los componentes de manera modular, de tal forma que cada vez que se añadía una nueva funcionalidad, se comprobaba su integración con el resto del sistema.

De esta manera se han podido detectar y corregir errores de integración en etapas tempranas.

8.4. Pruebas de Sistema

Las pruebas de sistema tienen como objetivo validar el comportamiento de la aplicación en un entorno cercano al despliegue real, incluyendo todos los componentes externos que intervienen en el proceso de análisis: base de datos PostgreSQL, motor ClamAV, servidor Apache Tika y motor de reglas YARA.

En este contexto se utilizan ficheros de prueba almacenados en la carpeta de muestras del proyecto. Estos incluyen:

- Documentos ofimáticos con macros maliciosas, utilizados para comprobar la extracción de metadatos y análisis de macros.
- Imágenes con y sin esteganografía, que sirven para validar los módulos de detección de esteganografía y la representación de diferencias en los informes.
- Ficheros de audio limpios y con esteganografía embebida, que permiten verificar la generación de espectrogramas y la detección de patrones anómalos.
- Ejecutables con características típicas de *malware*. Estos han sido recopilados de los laboratorios del libro Practical Malware Analysis (Sikorski y Honig, 2012) y han sido empleados para contrastar las detecciones de ClamAV y las reglas YARA.

8.5. Pruebas de Aceptación

Las pruebas de aceptación se diseñan desde la perspectiva del usuario final y tienen como objetivo comprobar que se cumplen los requisitos funcionales definidos.

A partir de los casos de uso identificados, se han realizado pruebas manuales que simulan escenarios típicos de uso de la aplicación. Por ejemplo:

- Un usuario autenticado sube un documento sospechoso, consulta el estado de su análisis, accede al informe detallado y toma decisiones a partir de las evidencias mostradas.
- Un administrador gestiona la base de usuarios, supervisa los *logs* del sistema para detectar comportamientos anómalos y revisa las alertas generadas.
- Un usuario revisa el histórico de ficheros subidos y elimina aquellos que ya no son relevantes.

Estos escenarios se han realizado utilizando un navegador web y el mismo servicio de producción.

8.6. Pruebas de Rendimiento

Además de validar la funcionalidad, se realizaron pruebas de rendimiento con el objetivo de comprobar el comportamiento del sistema bajo uso concurrente. Para ello se utilizó la herramienta Locust³, que permite simular usuarios que ejecutan de forma repetida el flujo completo de autenticación y subida de ficheros. Esta lógica de prueba se encuentra definida en el archivo `locustfile.py` dentro del directorio `load_tests`.

Cada usuario virtual ejecuta el siguiente escenario:

³Locust: <https://locust.io/>

1. Acceso a la página de inicio de sesión.
2. Envío del formulario de login con credenciales válidas.
3. Acceso a la página de subida de ficheros.
4. Subida de un documento de prueba que desencadena el análisis completo.

De esta forma se somete a pruebas de carga tanto la capa web como los motores de análisis internos.

Las pruebas se ejecutaron en un portátil con Windows 11, procesador de ocho núcleos y 16 GB de memoria RAM, levantando la aplicación en modo desarrollo y ejecutando Locust desde la misma máquina.

Para cada nivel de usuarios concurrentes se dejó estabilizar la carga durante varios minutos y se registraron las métricas reportadas por Locust (latencia media, percentil 95 % y peticiones por segundo). De forma paralela, se monitorizó el proceso de la aplicación mediante el uso de scripts de PowerShell ejecutando la función `Get-Process` que muestrea con periodicidad el uso de CPU y memoria del proceso `python.exe`.

La Tabla 8.1 recoge resultados aproximados representativos de estas ejecuciones.

Usuarios	Lat. media (ms)	Lat. 95 % (ms)	RPS	CPU (%)	Mem. (MB)
5	2 200	8 200	0,5	23	190
15	4 800	15 450	0,9	56	260
30	7 500	22 800	1,2	79	350

Tabla 8.1 - Resultados de las pruebas de rendimiento usando Locust

Como se puede observar, incluso en el escenario más exigente de 30 usuarios concurrentes, la aplicación mantiene un rendimiento cercano a 1,2 peticiones por segundo con un uso de CPU por debajo del 80 % y un consumo de memoria en torno a 300 MB. Estos resultados indican que la aplicación es capaz de manejar múltiples análisis concurrentes sin una degradación significativa del servicio.

9. Despliegue del Sistema

En este capítulo se describe la arquitectura física del sistema, las instrucciones de despliegue y las instrucciones de uso/mantenimiento del sistema.

9.1. Arquitectura Física

La arquitectura física de ForenAlyze se basa en una aplicación web desplegada sobre un servidor que expone el servicio al resto de usuarios a través de una URL. La herramienta está pensada para ser consumida desde un navegador web estándar y permite dos modos principales de despliegue:

1. Ejecución directa con Python: ejecutando la aplicación Flask desde un entorno Python local, instalando las dependencias descritas en el fichero `requirements.txt`.
2. Virtualización con contenedores: utilizando Docker¹ y Docker Compose para orquestar la aplicación web ForenAlyze, la base de datos PostgreSQL y el servidor Apache Tika.

En el escenario típico de producción, la arquitectura física se concreta en los siguientes componentes:

- Un contenedor para la aplicación web (Flask + Gunicorn) que recibe las peticiones HTTP de los usuarios.
- Un contenedor con PostgreSQL, que actúa como base de datos relacional para usuarios, ficheros analizados, resultados de análisis, alertas y *logs*.
- Un contenedor con Apache Tika, utilizado para la extracción de texto y metadatos de documentos.
- El motor antivirus ClamAV instalado en el contenedor de la aplicación, así como la integración con motores externos (YARA, VirusTotal, *sandbox* dinámico) accesibles a través de la red.

Para un uso individual o de laboratorio, el sistema puede ejecutarse en un único equipo (portátil o sobremesa), que aloje los anteriores servicios. A medida que aumente el número de usuarios concurrentes, el tamaño de los ficheros analizados o la intensidad de uso de los motores de análisis crecerán proporcionalmente los requisitos hardware.

Como referencia para un entorno de pruebas multiusuario se recomiendan las siguientes especificaciones mínimas:

- Procesador: 4 núcleos físicos (por ejemplo, Intel Xeon o equivalente).
- Memoria RAM: 16 GB.

¹Docker - <https://www.docker.com/>

- Almacenamiento: SSD de al menos 1 TB para alojar base de datos, ficheros subidos y copias de seguridad.

Inicialmente se valoró el despliegue de la aplicación en la nube mediante Azure App Service, aprovechando su plan gratuito para exponer una instancia de demostración. Sin embargo, las limitaciones de dicho plan hacían poco práctico ejecutar de forma sostenida todos los componentes necesarios.

Por este motivo, opté finalmente por utilizar la plataforma Replit²; cuyo plan gratuito ofrecía mayores recursos y una configuración más sencilla para este tipo de proyectos académicos.

Con el fin de facilitar la evaluación del proyecto, se ha desplegado una instancia pública de demostración. Esta instancia está accesible en la URL:

<https://foren-alyze--javivimol.replit.app/>

Debido a limitaciones de la propia plataforma Replit, no se encuentra disponible el motor antivirus ClamAV, aunque el resto de servicios de análisis sí están operativos.

Se trata de un entorno gratuito con recursos limitados, por lo que se recomienda utilizar ficheros de prueba de tamaño reducido; preferiblemente de los incluidos en el propio repositorio del proyecto en la carpeta *samples*, y evitar cargas masivas o análisis de grandes volúmenes de datos.

El usuario disponible para acceder a esta instancia es el siguiente:

- Usuario: `testNormalUser`
- Contraseña: `Normal_User.123!`

9.2. Instrucciones de despliegue

En esta sección se describen los pasos necesarios para instalar y ejecutar la aplicación sobre la infraestructura física descrita previamente, tanto mediante contenedores Docker como directamente con Python.

9.2.1. Ejecución con Python

ForenAlyze puede ejecutarse directamente sobre un entorno Python local.

Los requisitos básicos son:

- Python 3.11 (o versión compatible) instalado en el sistema.
- Acceso a una base de datos.
- Opcionalmente, ClamAV y Apache Tika instalados y accesibles para aprovechar todas las capacidades de análisis.

²Replit - <https://www.replit.com/>

El procedimiento típico de instalación sería:

1. Clonar el repositorio del proyecto desde GitHub.
2. Crear y activar un entorno virtual de Python.
3. Instalar las dependencias con `pip install -r requirements.txt`.
4. Configurar las variables de entorno mínimas definidas en `.env.example`, preferiblemente en un fichero `.env`.
5. Ejecutar el script `create_admin.py` para inicializar la base de datos y crear el usuario administrador.
6. Lanzar la aplicación con `python run.py` y acceder a `http://localhost:5000/`.

9.2.2. Virtualización

Como alternativa a la ejecución en un entorno Python y para simplificar el despliegue, hemos definido un entorno de ejecución basado en Docker y Docker Compose. Este enfoque permite levantar con un único comando; la aplicación web ForenAlyze, la base de datos PostgreSQL y el servidor Tika.

El despliegue parte de dos ficheros principales: `Dockerfile` y `docker-compose.yml`.

- **Dockerfile:** parte de una imagen base `python:3.11-slim`, instala las dependencias de sistema necesarias, copia el código fuente del proyecto y ejecuta `pip install -r requirements.txt` para instalar todas las dependencias Python. Finalmente, configura el contenedor para arrancar la aplicación ejecutando primero el script `create_admin.py` y después el servidor WSGI Gunicorn³ definido en `run.py`.
- **docker-compose.yml:** orquesta tres servicios:
 1. **postgres:** contenedor con PostgreSQL 16.
 2. **tika:** contenedor con Apache Tika Server.
 3. **web:** contenedor que construye la imagen de ForenAlyze usando el `Dockerfile`.

Para construir y lanzar el sistema mediante contenedores, basta con situarse en el directorio raíz del proyecto y ejecutar:

```
docker compose up --build
```

Tras unos minutos, la aplicación estará disponible en `http://localhost:8000/`.

9.2.3. Requisitos previos

Para los usuarios finales del sistema se recomienda el uso de navegadores web modernos y actualizados. A modo orientativo:

³Gunicorn - <https://gunicorn.org/>

- Google Chrome 100 o superior.
- Mozilla Firefox 100 o superior.
- Microsoft Edge 98 o superior.
- Opera 80 o superior.
- Safari 13 o superior.

Para la instalación y funcionamiento del sistema se requieren los siguientes componentes:

- Docker 20 o superior y Docker Compose (para el despliegue basado en contenedores).
- Python 3.11 con las dependencias descritas en el anexo B.
- Acceso a red hacia los servicios externos opcionales que se deseen utilizar (VirusTotal, Hybrid Analysis...), en caso de habilitar dichas integraciones.

9.2.4. Inventario de componentes

El proyecto incluye los siguientes componentes principales:

- Aplicativo web para el análisis forense automatizado de documentos y ficheros, desarrollado con Flask.
- Scripts de inicialización y utilidades.
- Definiciones de contenedores y orquestación en los ficheros de Docker
- Módulo de integración con motores de análisis: ClamAV, YARA, Apache Tika, VirusTotal y *sandbox* dinámico.
- Conjunto de reglas YARA almacenadas en el subdirectorio `yara-signature-base`, con firmas para múltiples familias de *malware*.
- Carpeta `samples` con ficheros de ejemplo, para validar el funcionamiento del sistema.
- Ficheros de configuración y plantillas.

9.2.5. Procedimientos de instalación

El código fuente de ForenAlyze se encuentra disponible públicamente en GitHub en la siguiente URL:

<https://github.com/Javivi-MR/ForenAlyze>

Instalación con Docker.

1. Clonar el repositorio en el equipo donde se vaya a desplegar el sistema.
2. Crear un fichero `.env` en el directorio raíz con las variables de entorno necesarias.
3. Desde el directorio raíz del proyecto, ejecutar:

```
docker compose up --build
```

4. Esperar a que Docker construya la imagen de la aplicación, arranque los contenedores de PostgreSQL, Tika y ForenAlyze, y ejecute el script de creación del usuario administrador.
5. Acceder a la aplicación desde un navegador en <http://localhost:8000/>.

Instalación con Python.

1. Clonar el repositorio del proyecto.
2. Crear y activar un entorno virtual de Python 3.11.
3. Instalar las dependencias:

```
pip install -r requirements.txt
```

4. Definir las variables de entorno mínimas, ya sea exportándolas en el sistema o utilizando un fichero `.env`. Si no se configura una base de datos externa, la aplicación usará SQLite por defecto.
5. Inicializar la base de datos y el usuario administrador:

```
python create_admin.py
```

6. Lanzar la aplicación Flask:

```
python run.py
```

7. Acceder a la aplicación en <http://localhost:5000/>.

9.2.6. Pruebas de implantación

Para verificar que el sistema se ha desplegado correctamente y que los componentes principales funcionan como se espera, se proponen el siguiente conjunto de acciones de validación:

1. Acceder a la URL del sistema en el despliegue con Docker o la URL pública en Replit desde un navegador web y comprobar que se muestra la página principal.

2. Acceder a la sección de *Login* e iniciar sesión con las credenciales del usuario administrador creado por `create_admin.py`. Verificar que se accede correctamente al *dashboard*.
3. Crear un usuario normal desde la sección de administración de usuarios y comprobar que la operación finaliza con éxito.
4. Cerrar sesión e iniciar sesión con el nuevo usuario creado.
5. Subir un fichero de prueba desde la sección de subida y verificar que se genera un análisis, que se muestra un resumen en el panel de control y que aparece una alerta indicando que el informe está disponible.
6. Consultar el informe de análisis correspondiente y comprobar que se muestran los metadatos básicos, los hashes del fichero y el veredicto final.
7. Acceder al apartado de *logs* y verificar que se han registrado entradas asociadas a las acciones realizadas.
8. Crear un segundo usuario normal, subir un fichero con cada uno de ellos y verificar en la sección de almacenamiento, que cada usuario sólo puede ver y gestionar sus propios ficheros.
9. Iniciar sesión como administrador e intentar eliminar un usuario normal desde la interfaz de administración; comprobando que se borran correctamente sus ficheros, análisis, alertas y *logs* asociados.

Si las acciones anteriores se ejecutan sin errores y los resultados observados son los esperados, el sistema se ha instalado correctamente y está listo para utilizarse.

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio

Para mantener el sistema operativo y minimizar la pérdida de información, es suficiente con definir una política básica de copias de seguridad y realizar un mínimo de tareas de mantenimiento preventivo.

En el despliegue recomendado con Docker, los datos se concentran en dos ubicaciones: el volumen de PostgreSQL (que almacena usuarios, ficheros, análisis, alertas y *logs*) y el directorio de ficheros subidos (`instance/uploads/`) dentro del contenedor de la aplicación. En entornos de desarrollo que utilizan SQLite, la base de datos reside en un fichero dentro del directorio `instance/`.

Se recomienda:

- Realizar una copia de seguridad completa de la base de datos al menos una vez por semana y conservar copias incrementales, o diferenciales con mayor frecuencia según el volumen de cambios.
- Revisar periódicamente el directorio de ficheros subidos, manteniendo la coherencia entre los identificadores almacenados en base de datos y los ficheros físicos.

- Vigilar el espacio disponible en disco y, si es necesario, ajustar las cuotas de almacenamiento por usuario.
- Mantener actualizadas las firmas de ClamAV y las reglas YARA.
- Revisar de forma periódica los *logs* de aplicación para detectar errores recurrentes o intentos de acceso no autorizado.

Parte III

Epílogo

10. Conclusiones y trabajo futuro

En este capítulo se presentan las conclusiones obtenidas a partir del desarrollo del proyecto, así como las posibles líneas de trabajo futuro que podrían derivarse de este trabajo.

10.1. Conclusiones

En este TFM se ha diseñado e implementado una aplicación web, denominada ForenAlyze, para el análisis forense automatizado de ficheros; centrada en documentos ofimáticos, imágenes, audio y ejecutables. ForenAlyze permite registrar usuarios, gestionar diferentes roles (usuario normal y administrador), iniciar sesión de forma segura y subir ficheros para su análisis. Cada fichero se somete a un *pipeline* que integra múltiples motores y técnicas: antivirus ClamAV, reglas YARA, análisis de macros, detección de esteganografía en imágenes y audio, extracción de texto y metadatos mediante Apache Tika, consultas a VirusTotal y análisis dinámico en *sandbox*. Los resultados se presentan en informes y en un panel de control con alertas, estadísticas y registros de actividad.

También se ha construido toda la infraestructura necesaria en torno a este núcleo de análisis: modelos de datos para usuarios, archivos, análisis, alertas y registros; vistas para la gestión del almacenamiento; un sistema de cuotas por usuario, y un módulo de administración que permite gestionar cuentas, revisar registros y eliminar usuarios junto con toda su información asociada. El sistema incluye pruebas automatizadas con *pytest*, integración continua mediante GitHub Actions y un despliegue reproducible basado en Docker y Docker Compose, así como una instancia de demostración pública en Replit.

Los resultados demuestran que ForenAlyze es una herramienta de código abierto que facilita y hace posible la integración de múltiples técnicas de análisis forense en una única plataforma, reduciendo el esfuerzo manual necesario y mejorando la calidad de los veredictos obtenidos, sentando una base sólida para futuros trabajos en el ámbito de la ciberseguridad.

10.2. Lecciones aprendidas

Más allá de estos objetivos técnicos, el desarrollo de ForenAlyze me ha permitido profundizar de forma significativa en varios ámbitos: el análisis forense de *malware* y documentos (estáticos y dinámicos), la construcción de aplicaciones web seguras con Flask (autenticación, control de acceso, protección frente a ataques web comunes), el diseño de *pipelines* de análisis modulares y extensibles, y el despliegue de servicios en contenedores y en la nube. Este aprendizaje transversal, ha sido tan importante como

el propio producto final y sienta una base sólida para futuros trabajos en el ámbito de la ciberseguridad.

Desde el inicio del proyecto ha sido necesario estudiar en profundidad el problema de la detección y análisis de *malware*, así como las técnicas clásicas y modernas que se emplean en este campo. La revisión de la literatura y de referencias especializadas como (Sikorski y Honig, 2012) me ha ayudado a comprender mejor la diferencia entre análisis estático y dinámico, la importancia de los indicadores de compromiso (IOC), las técnicas de ofuscación habituales y el papel que juegan las herramientas de automatización en el trabajo diario de un analista.

En el plano tecnológico, he aprendido a combinar de forma coherente distintos motores y fuentes de información: motores de firmas (ClamAV, YARA), extracción de metadatos (Tika), servicios externos (VirusTotal, *sandbox*) y heurísticas (detección de esteganografía, análisis de audio). Integrar todos estos componentes en un único *pipeline* robusto, manejando errores y tiempos de respuesta, ha sido un ejercicio práctico de ingeniería de software que me ha obligado a cuidar la modularidad, el registro de información y la trazabilidad de los resultados.

También he consolidado conocimientos avanzados de desarrollo web con Flask, de bases de datos y migraciones, de orquestación con Docker y de prácticas de calidad como las pruebas automatizadas y la integración continua. En conjunto, el proyecto ha supuesto un recorrido completo desde la concepción de la idea hasta un sistema funcional desplegado en producción.

10.3. Trabajo futuro

El proyecto abre múltiples líneas de mejora y ampliación que podrían explorarse en trabajos posteriores. A continuación se destacan algunas propuestas que, además, se apoyan en trabajos académicos recientes para servir como base teórica y práctica:

- Integración de un módulo de clasificación basado en *machine learning*. En la versión actual, el veredicto final de ForenAlyze se obtiene a partir de reglas y umbrales construidos sobre motores de firmas, heurísticas y puntuaciones externas. Una posible mejora sería incorporar un clasificador entrenado con las propias características que ya calcula el sistema (hashes, metadatos, resultados de YARA, estadísticas de VirusTotal, puntuaciones de *sandbox*), de forma que pueda aprender patrones más complejos y generalizables. Trabajos recientes centrados en la detección y clasificación de *malware* mediante algoritmos de aprendizaje automático y *pipelines* de preprocesado y selección de características (Dominguez Mellado, 2025), así como encuestas sobre técnicas de clasificación de *malware* basadas en *machine learning* (Ucci et al., 2019), proporcionan una base sólida para diseñar este tipo de modelos y comparar distintos algoritmos.
- Modelos de *deep learning* especializados en ficheros binarios. Otra mejora sería evaluar redes neuronales profundas aplicadas directamente sobre el binario o sobre representaciones intermedias (Saxe y Berlin, 2015). Por ejemplo, se podría utilizar

un modelo que procesa el ejecutable completo como una secuencia de bytes ([Raff et al., 2017](#)).

- Enriquecimiento del conjunto de reglas y de la inteligencia de amenazas. Actualmente el sistema se apoya en un conjunto de reglas YARA externas y en las firmas de ClamAV. Una posible ampliación podría centrarse en la gestión avanzada de estas reglas: actualización continua desde repositorios comunitarios y la correlación con fuentes de inteligencia de amenazas (feeds de IOC, listas de dominios y direcciones IP maliciosas).
- Clustering y priorización de incidentes. A medida que aumente el volumen de ficheros analizados, será útil contar con mecanismos que agrupen muestras similares (familias de *malware*, técnicas de ofuscación, comportamiento en *sandbox*) y ayuden a priorizar los casos más relevantes. Siguiendo trabajos que combinan características estáticas y dinámicas para agrupar muestras ([Ucci et al., 2019](#)), se podría incorporar módulos de clustering y *scoring* que orienten al analista hacia los incidentes más críticos.
- Ampliación de la cobertura de formatos y técnicas de análisis. Finalmente, el sistema podría extenderse para soportar nuevos tipos de ficheros y técnicas avanzadas de análisis. Entre otras líneas, cabría profundizar en el tratamiento de documentos ofimáticos (analizando no sólo macros sino también objetos embebidos complejos), incorporar soporte específico para otros formatos ejecutables de Windows como librerías dinámicas (.dll) y paquetes comprimidos.

El desarrollo de estas líneas no solo incrementaría el alcance y la utilidad práctica de ForenAlyze, sino que también permitiría seguir profundizando en temas de gran interés actual en ciberseguridad; como la detección inteligente de *malware*, la automatización del análisis forense y la integración de múltiples fuentes de datos en sistemas de apoyo a la toma de decisiones.

Información sobre Licencia

Información sobre Licencia

“*ForenAlyze: Plataforma modular para la recogida y análisis automatizado de evidencias forenses digitales*” es un software de código abierto distribuido bajo la Licencia Pública General de GNU versión 3.0¹.

Esto significa que deben cumplirse los siguientes derechos y obligaciones:

- Tienes derecho a usar, copiar, modificar, fusionar, publicar y distribuir este software
- Si realizas una modificación del software, debe de estar distribuida bajo la licencia GPL - 3.0
- Si distribuyes este software, debes proporcionar acceso al código fuente completamente.

¹GPL - 3.0 - <https://www.gnu.org/licenses/gpl-3.0.html>

Bibliografía

- Alenezi, M. N., Alabdulrazzaq, H. K., Alshaher, A. A., y Alkharang, M. M. (2022). Evolution of malware threats and techniques: a review. *International Journal of Communication Networks and Information Security (IJCNIS)*, 12(3).
- Almuhtadi, W., Bahri, S., Fenwick, W., Henderson, L., Henley-Vachon, L., y Mukasa, J. (2021). Malware detection and security analysis capabilities in a continuous integration / delivery context using assemblyline. In *2021 IEEE International Conference on Consumer Electronics (ICCE)*, pages 1–5.
- Canadian Centre for Cyber Security (2020). Assemblyline 4: Scalable file triage and malware analysis. https://cybercentrecanada.github.io/assemblyline4_docs/.
- Carvey, H. y Altheide, C. (2011). *Digital Forensics with Open Source Tools*. Elsevier. Google-Books-ID: J8h8VWUmDuYC.
- Domínguez Mellado, A. (2025). Sistema de detección y clasificación de malware utilizando algoritmos de machine learning y fuentes de datos diversas. Repositorio institucional de la universidad de cádiz, Universidad de Cádiz.
- Fakhouri, H. N., AlSharaiah, M. A., Al hwaitat, A. k., Alkalaileh, M., y Dweikat, F. F. (2024). Overview of challenges faced by digital forensic. In *2024 2nd International Conference on Cyber Resilience (ICCR)*, pages 1–8.
- Guarino, A. (2013). Digital forensics as a big data challenge. In Reimer, H., Pohlmann, N., y Schneider, W., editors, *ISSE 2013 Securing Electronic Business Processes: Highlights of the Information Security Solutions Europe 2013 Conference*, pages 197–203. Springer Fachmedien Wiesbaden.
- K. O'Reillyt and A. Brukhovetsky (2024). Cape: Malware configuration and payload extraction. <https://github.com/kevoreilly/CAPEv2>. Proyecto open-source de análisis dinámico de malware.
- Knöchel, M. y Karius, S. (2024). Text steganography methods and their influence in malware: A comprehensive overview and evaluation. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security, IH&MMSec '24*, pages 113–124, New York, NY, USA. Association for Computing Machinery.
- Mahdi, R. H. y Trabelsi, H. (2024). Detection of malware by using yara rules. In *2024 21st International Multi-Conference on Systems, Signals & Devices (SSD)*, pages 1–8.
- Michelet, G., Breitingner, F., y Horsman, G. (2023). Automation for digital forensics: Towards a definition for the community. *Forensic Science International*, 349:111769.
- Pandey, S. K. y Mehtre, B. (2014). Performance of malware detection tools: A comparison. In *2014 IEEE International Conference on Advanced Communications, Control and Computing Technologies*, pages 1811–1817.

- Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., y Nicholas, C. K. (2017). Malware detection by eating a whole EXE. In *AAAI Workshop on Artificial Intelligence for Cyber Security (AICS)*. Preprint disponible en arXiv:1710.09435.
- Saxe, J. y Berlin, K. (2015). Deep neural network based malware detection using two dimensional binary program features. In *2015 10th International Conference on Malicious and Unwanted Software (MALWARE)*, pages 11–20.
- Sikorski, M. y Honig, A. (2012). *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, San Francisco, CA.
- UCA (2025). Tabla de presupuestos Capítulo VI. <https://ugi.uca.es/wp-content/uploads/2025/01/DEF.-Costes-aproximados-contratacion-2025.pdf?u>.
- Ucci, D., Aniello, L., y Baldoni, R. (2019). Survey of machine learning techniques for malware analysis. *Computers & Security*, 81:123–147.

Parte IV

Anexos

A. Manual de usuario

A.1. Introducción

ForenAlyze es una aplicación web orientada al análisis forense automatizado de ficheros potencialmente maliciosos. El sistema permite a los usuarios subir documentos ofimáticos, imágenes, ficheros de audio y ejecutables para que sean analizados de forma centralizada.

A partir de cada fichero subido, ForenAlyze ejecuta un *pipeline* de análisis que combina distintos motores y fuentes de información (ClamAV, reglas YARA, análisis de macros, detección de esteganografía, extracción de metadatos con Apache Tika, consultas a VirusTotal y análisis dinámico en *sandbox*). Finalmente, el usuario puede consultar un informe detallado con los resultados y un veredicto global, así como acceder a un panel con estadísticas, alertas y registros de actividad.

A.2. Instalación

Para probar el sistema no es estrictamente necesario realizar ninguna instalación local; se dispone de una instancia de demostración pública cuyo enlace se indica en la sección 9.1. Accediendo a dicha URL, el usuario puede utilizar ForenAlyze desde su navegador; respetando las limitaciones propias del entorno de pruebas.

En caso de querer instalar y desplegar el aplicativo de forma local, el usuario deberá seguir las instrucciones descritas en la sección 9.2. De forma resumida, existen dos opciones:

- Despliegue con Docker y Docker Compose, que levanta la aplicación web, la base de datos PostgreSQL y el servidor Apache Tika mediante contenedores.
- Ejecución directa con Python, instalando las dependencias de `requirements.txt`, configurando las variables de entorno necesarias y ejecutando `create_admin.py` seguido de `run.py`.

En ambos casos, una vez completada la instalación, el usuario puede acceder a ForenAlyze desde un navegador web introduciendo la URL correspondiente.

A.3. Uso del sistema

En esta sección se describen las principales acciones que puede realizar un usuario en el sistema.

A.3.1. Inicio del sistema

Al acceder por primera vez a la aplicación, el usuario se sitúa en la página de inicio. Esta pantalla muestra una breve descripción del proyecto, sus objetivos y un resumen de las capacidades de análisis que ofrece la herramienta.

En la barra de navegación superior se encuentran los accesos a:

- La página principal.
- La sección de inicio de sesión.
- El panel de control (*dashboard*) y las secciones de análisis (disponibles tras iniciar sesión).
- La sección de información *Acerca de*, con detalles técnicos del proyecto.

A.3.2. Inicio de sesión

Al seleccionar la opción de inicio de sesión, se muestra un formulario donde el usuario debe introducir su nombre de usuario y contraseña. Tras autenticarse correctamente, se podrá acceder al panel principal del sistema.

Existen dos tipos de usuarios:

- Usuario normal, que puede subir ficheros, consultar sus propios análisis, revisar sus alertas y gestionar su espacio de almacenamiento.
- Administrador, que además puede gestionar usuarios, revisar los logs globales y supervisar el estado general del sistema.

Las cuentas se crean habitualmente desde el panel de gestión de usuarios.

A.3.3. Panel de control y alertas

Una vez iniciada la sesión, el usuario accede al panel de control (*dashboard*). En esta pantalla se muestran, entre otros elementos:

- Un listado de los análisis recientes realizados por el usuario (y, en el caso de administradores, una visión global).
- Un resumen de veredictos representado en forma de tarjetas o gráficos.
- Alertas pendientes.

Desde este panel, el usuario puede navegar rápidamente a los informes de análisis, al histórico de ficheros y a otras secciones del sistema.

A.3.4. Subida de ficheros

Para analizar un nuevo fichero, el usuario debe dirigirse a la sección de subida. Al hacerlo, se presenta un formulario que permite seleccionar un archivo local.

Las principales consideraciones son:

- Se debe subir únicamente ficheros soportados por el sistema.
- Dependiendo de la configuración del despliegue, puede existir un tamaño máximo permitido por fichero y una cuota de almacenamiento por usuario.
- Cada fichero subido se asocia al usuario autenticado en ese momento y sólo será visible para él (excepto para los administradores).

Tras confirmar la subida, el sistema registra el fichero y lanza el *pipeline* de análisis de forma automática. El usuario recibirá una alerta cuando el informe esté listo.

A.3.5. Histórico de ficheros y análisis

La sección de ficheros y análisis muestra el listado de todos los ficheros que el usuario ha subido al sistema, junto con información resumida de cada uno:

- Nombre original del fichero.
- Tipo y tamaño.
- Fecha de subida y fecha de análisis.
- Veredicto final del análisis.

Desde esta lista el usuario puede:

- Acceder al informe de análisis completo de un fichero concreto.
- Eliminar ficheros que ya no necesite, liberando espacio de almacenamiento.

A.3.6. Informe de análisis

Al seleccionar un análisis concreto, el sistema muestra un informe detallado con los resultados obtenidos. Este informe incluye, entre otros elementos:

- Información básica del fichero: nombre, tipo, tamaño y ruta de almacenamiento interna.
- Hashes calculados (MD5, SHA1, SHA256).
- Veredicto global y resumen de los motores activados (ClamAV, YARA, macros, esteganografía, VirusTotal y *sandbox*).
- Detalles de detecciones YARA (reglas que han coincidido y sus metadatos).
- Información de macros sospechosas en documentos Office, si las hubiera.

- Resultados de análisis de esteganografía en imágenes y audio.
- Extractos de texto y metadatos relevantes obtenidos mediante Apache Tika.
- Enlaces y resúmenes de análisis dinámicos.

Cuando el propietario del fichero consulta el informe, las alertas asociadas a ese análisis se marcan como leídas y el contador de notificaciones pendientes se actualiza.

A.3.7. Gestión de almacenamiento

En la sección de almacenamiento el usuario puede consultar el espacio ocupado por sus ficheros y la cuota total asignada. Esta pantalla muestra:

- Un indicador del porcentaje de almacenamiento utilizado.
- El listado de ficheros asociados al usuario con su tamaño.

Desde aquí el usuario puede eliminar ficheros antiguos o no necesarios para liberar espacio. Los administradores, pueden revisar el uso de almacenamiento agregado por usuario y detectar situaciones en las que se requiera ampliar recursos.

A.3.8. *Logs* y auditoría

La sección de *logs* permite revisar los registros de actividad del sistema. Un usuario normal verá únicamente los eventos asociados a su cuenta (inicios de sesión, subidas de ficheros, consultas de informes. . .), mientras que un administrador puede visualizar el log completo, incluyendo eventos del sistema y de otros usuarios.

Esta funcionalidad es útil para:

- Auditar acciones realizadas en el sistema.
- Investigar incidencias o usos indebidos.
- Verificar que se han ejecutado correctamente determinadas operaciones.

A.3.9. Gestión de usuarios

Los usuarios con rol de administrador disponen de una sección adicional de gestión de usuarios. Desde ella pueden:

- Crear nuevas cuentas de usuario, asignándoles nombre de usuario, correo electrónico y rol.
- Modificar datos básicos de cuentas existentes.
- Desactivar o eliminar usuarios que ya no deban tener acceso al sistema, eliminando así todos los datos asociados a esa cuenta.

Estas operaciones están protegidas de forma que un administrador no puede eliminar su propia cuenta ni borrar a otros administradores.

A.3.10. Perfil de usuario

Finalmente, el usuario puede acceder a su perfil para revisar o actualizar algunos de sus datos como nombre, foto de perfil o contraseña.

B. Dependencias del aplicativo

Librería	Compatibilidad	Versión
Flask	exacta	2.3.2
Flask-Login	exacta	0.6.3
Flask-WTF	exacta	1.1.1
Flask-SQLAlchemy	exacta	3.0.5
Flask-Migrate	mínima	4.0.5
WTForms	exacta	3.1.2
Werkzeug	exacta	2.3.7
python-dotenv	mínima	1.0.0
Pillow	mínima	10.0.0
mutagen	mínima	1.47.0
yara-python	mínima	4.3.0
requests	mínima	2.31.0
oletools	mínima	0.60
numpy	mínima	1.26.0
matplotlib	mínima	3.8.0
psycpg2-binary	mínima	2.9.9
reportlab	mínima	4.0.7
gunicorn	mínima	21.2.0
alembic	mínima	1.13.0
pytest	mínima	7.4.0
locust	mínima	2.20.0

Tabla B.1 - Tabla de dependencias de ForenAlyze

